2 目的最短路問題に対する弱パレートパスの全列挙アルゴリズム

李 智雄　　南川 智都　　奥野 貴之
電気通信大学　静岡理工科大学　成蹊大学

和文概要

本研究では，2 目的最短路問題の弱パレート解の列挙を考える．既存研究の多くは，辺コストの非負性や目的関数が負となる閉路の非存在を仮定している．これに対し，本研究では，負のコストや少なくとも一方の目的関数が負となる閉路を含む一般の有向グラフを対象とし，ノードの重複を許さない単純パスの中からすべての弱パレート解を列挙する，ラベル修正法に基づく新たなアルゴリズム WP-LC を提案する．WP-LC は既存のラベル修正法とは異なり，負の辺コストや目的関数値が負となる閉路を含むグラフにおいても，有限回の反復で単純パス中の全ての弱パレート解を出力することが理論的に保証される．さらにこの手法は，探索過程における動的な閉路検知に加え，事前探索に基づく枝刈り，有望なパスを優先的に探索するヒューリスティクス，および解の欠落を防ぎながら不要なパスを削減する独自のラベル削除機構を備えている．このような高速化によって，サイズ 100×100 のグリッドグラフやノード数 100 万程度のベンチマーク問題に対して，Sedeño-Noda と Colebrook の手法を元に実装したラベル設定法よりも高速に弱パレート解の列挙ができることを，数値実験によって示す．

キーワード: 離散最適化，アルゴリズム，最短路問題，多目的最適化

1. はじめに

最短路問題とは，各辺に付与された時間や運賃などの単一コストの合計が最小となるパスを探索する問題であり，その応用範囲は極めて広い．例えば，カーナビゲーションシステムや乗換案内といった交通ネットワークへの適用はもちろんのこと，工場内における搬送ロボットの動作計画，物流における配送計画問題，さらには都市計画における災害時の避難パスの策定など，現代の社会基盤を支える多くのシステムにおいて，その基盤技術として活用されている．このように，最短路問題は実社会における様々な応用例をもつほか，問題の離散構造を活かした Dijkstra 法 [3] や Bellman–Ford 法 [1] といった多項式時間アルゴリズムが知られており，典型的な離散最適化問題の一種である．

しかし，現実社会における意思決定や経路探索では，考慮すべき基準が「時間」や「運賃」といった単一の指標だけであるとは限らない．例えば，移動時間を最小化しつつ運賃も抑えたい場合や，距離を短くしたいが道路の安全性も確保したい場合など，ある目的を改善すれば別の目的が悪化するというトレードオフの関係にある複数の目的を同時に考慮しなければならない場面が多々存在する．このように，複数の目的関数を持つ問題は多目的最短路問題 (Multi-objective Shortest Path Problem) と呼ばれ，その中でも特に目的が 2 つの場合を 2 目的最短路問題 (Bi-objective Shortest Path Problem) という．本研究では，この 2 目的最短路問題を扱う．なお，区別のために，通常の辺上のコストが 1 つの問題を単目的最短路問題 (Single-Objective Shortest Path Problem) と呼ぶ．

2 目的最短路問題も単目的最短路問題と同様，目的関数の最小化を目指す．2 目的最短路問題においては，一般に，すべての目的関数を同時に最小化する解が存在するとは限らず，一方の目的関数を改善しようとするともう一方の目的関数が悪化してしまう状況が想定さ

れる．このような目的関数間にトレードオフの関係があることが一般的で，二つの目的関数を同時に最小化することは困難な場合が多い．このため，2 目的最適化では，パレート最適性というある種の最適性を持ったパレート解と呼ばれる解を求めることが一般的である．パレート解とは，ある特定の目的関数を改善するためには他の目的関数を悪化させざるを得ない解のことである．

2 目的最短路問題の研究が本格的にはじまったのは 1980 年頃で，初期のアルゴリズムの研究としては [7, 9] が挙げられる．これらのアルゴリズムはラベリング法と呼ばれ，各ノードに対して，暫定的な目的関数値を「ラベル」として保持・更新する．ラベリング法は，一度設定したラベルが後から変更されないように設定を行う「ラベル設定法」と，アルゴリズム終了まで逐次的に更新される可能性がある「ラベル修正法」の 2 つに分類される．現在，高速に動作する代表的なラベル設定法として Sedeño-Noda ら [13] により提案された単目的 Dijkstra 法の拡張版アルゴリズムが知られており，すべてのパレート解が列挙できることが保証されている．一方でラベル修正法に関する初期の重要研究として [11] があり，両手法の比較実験を行い，インスタンスによって高速に動作するアルゴリズムが異なると報告している．多目的最短路問題について比較実験を行った論文として [6] も知られており，SLF（Small Label First）と呼ばれる効率的なデータ更新方法を提案している．現在高速に動く代表的なラベル修正法は [8] である．ラベル修正法については現在も [8] のような研究が継続的に行われている．一方で，著者らの知る限り，負閉路を許す一般の有向グラフにおいて，単純パスに限定したパレート解の全列挙を保証するラベル修正法は知られていない．留意すべき点として，Sastry ら [12] は単目的 Bellman–Ford 法の拡張版アルゴリズムであるラベル修正法を提案しており，グラフに負のコストを持つ辺が存在する場合でもパレート解を列挙することが可能である．ところが，このアルゴリズムは負閉路の存在を許しておらず，事前に各目的ごとのグラフにおける負閉路の検知を行っている．なお，本章で紹介した以外の多目的最短路問題の文献も豊富に存在し，2024 年の調査論文 [2] の 2.2.1 章にまとめられている．

さて，本研究では応用上の観点から，弱パレート解を効率的に列挙することを目指す．弱パレート解とは，すべての目的関数を同時に改善することはできない解である．定義上，パレート解は弱パレート解に含まれる．パレート解だけでなく，弱パレート解を網羅的に列挙しておくことは，実運用上の対応策の柔軟性を高める上で重要である．特に交通ネットワークや災害時の避難パスの設計においては，あるルートが突如利用不可能になったり，コストが急増したりする事態が想定される．そのような場合，あらかじめ求めておいた弱パレートフロンティアが有効な代替案を提供し得る． さらに，本問題の解は離散的な点集合となるため，弱パレート解集合の全体像を直感的に把握することは容易ではない．弱パレート解を含めて広く列挙することで，解の分布や傾向をより明確に捉え，問題構造の深い理解にも繋がると期待される．

本研究では，負重みを許した有向グラフ上において単純パス制約をもつ 2 目的最短路問題を考える．単純パス制約がなければ，後述する非正閉路の影響により互いに強支配されないパスが無限に存在する場合があり，それらは殆どの場合非実用的である．一方で，同問題に対して単純パス制約下で弱パレート解を求めることには実用的意義がある．本問題に対し，新たなラベル修正法である WP-LC を提案し，正当性の証明を行った．このアルゴリズムで特筆すべき点は以下の通りである．

- 一般的なラベル修正法における支配関係を用いた緩和処理を強支配関係に変更

- 探索過程における動的な閉路の検知とパス履歴の包含関係に基づくラベル削除を行い，グラフ上に負閉路が存在する場合でも，それを検知するだけでなく，単純パス制約を課した同 2 目的最短路問題に対する弱パレート解の全列挙を保証
- 下界に関する枝刈り [13, 5] を用いた無駄なラベル伝播防止による高速化・省メモリ化
- ヒューリスティックに基づいたラベル処理の優先順位設定のため，未処理ラベル保存のためのデータ構造 SLF（Small Label First） [6] の拡張版の導入

本研究では提案手法 WP-LC の有用性を検証するために，既存のラベル設定法 [13] における支配関係の緩和処理を強支配関係に変更したアルゴリズムを実装し，数値実験を通して比較を行った．その結果，正重みグリッドグラフおよび大規模道路ネットワークにおいて，WP-LC は弱パレートパス列挙用に拡張した修正版ラベル設定法より短い平均計算時間を示した．また，大規模道路ネットワークでは，WP-LC の方が制限時間内に解決できたインスタンスが多いことを確認した．

本論文は以下のような構成になっている．第 2 章では，本論文で用いる記号や用語の定義を行う．第 3 章では，2 目的最短路問題の定義を説明し，パレート解および弱パレート解の性質について述べる．第 4 章では，本研究で新たに提案した弱パレート解列挙アルゴリズム WP-LC の詳細を説明する．第 5 章では，第 4 章で提案したアルゴリズムを実装し，既存手法（BDijkstra 法 [13] や，その弱パレート解への拡張版アルゴリズム）との比較実験を行った結果を示す．グリッドグラフや大規模道路ネットワークを用いた数値実験により，提案手法の計算時間や停止性に関する評価を行い，結果に対する考察を述べる．最後に，第 6 章で本論文の結論と今後の課題を述べる．

2. 準備

本研究では全体を通して有向グラフ $G = (V, A)$ を考える．ネットワーク $G = (V, A)$ に対して，以下の記号を定義する．ノード集合を $V = \{1, 2, \cdots, n\}$ とし，有向辺の集合を $A(|A| = m)$ とする．本論文では，同一の始点と終点を持つ複数の有向辺を含まないグラフを考える．ノード $v_1, v_2, \ldots, v_k$ とそれらをつなぐ有向辺 $(v_1, v_2), (v_2, v_3), \ldots, (v_{k-1}, v_k)$ が交互に現れる列

$$v_1, (v_1, v_2), v_2, (v_2, v_3), v_3, \ldots, v_{k-1}, (v_{k-1}, v_k), v_k$$

をパスと呼び，ノード列 $(v_1, v_2, \ldots, v_k)$ によって表現する．パスはノード列に同じノードを含むことを認め，同じノードが 2 回以上現れないパスを単純パスとよんで区別する．パス $P = (v_1, v_2, \ldots, v_k)$ 上の 2 ノード v_i, v_j $(1 \leq i \leq j \leq k)$ に対し，その間のパス $P_{ij} = (v_i, v_{i+1}, \ldots, v_j)$ をパス P の部分路と呼ぶ．また，パス P が部分路 P_1 と P_2 の結合で表されるとき，$P = P_1 + P_2$ と表記する．

本研究で多目的最短路問題を扱うにあたり，一般的な多目的最適化問題

$$\underset{\boldsymbol{x} \in X}{\text{minimize}}\ \boldsymbol{f}(\boldsymbol{x}) = \left(f^1(\boldsymbol{x}), f^2(\boldsymbol{x}), \cdots, f^q(\boldsymbol{x})\right)$$

に関して用いられる用語を導入する．q は目的関数の数である．また，本論文におけるベクトル間の不等号 $\leq$ および $<$ は成分ごとの不等号を表す．すなわち，$a \leq b$ はすべての成分 i について $a_i \leq b_i$ が成り立つことを意味し，$a < b$ はすべての成分 i について $a_i < b_i$ が成り立つことを意味する．

定義 2.1 (支配・強支配). 実行可能解 $\boldsymbol{x}, \boldsymbol{x'} \in X$ に対して，$\boldsymbol{f}(\boldsymbol{x}) \leq \boldsymbol{f}(\boldsymbol{x'})$ かつ $\boldsymbol{f}(\boldsymbol{x}) \neq \boldsymbol{f}(\boldsymbol{x'})$ ならば，$\boldsymbol{x}$ が $\boldsymbol{x'}$ を支配する（$\boldsymbol{x} \prec \boldsymbol{x'}$）といい，$\boldsymbol{f}(\boldsymbol{x}) < \boldsymbol{f}(\boldsymbol{x'})$ ならば，$\boldsymbol{x}$ が $\boldsymbol{x'}$ を強支配する（$\boldsymbol{x} \prec_{\mathrm{str}} \boldsymbol{x'}$）という．

定義 2.2 ((弱) パレート解). 実行可能領域 X において，$\boldsymbol{x_p}$ を支配するような $\boldsymbol{x}$ が存在しない場合，$\boldsymbol{x_p}$ をパレート解と呼び，$\boldsymbol{x_{wp}}$ を強支配するような $\boldsymbol{x}$ が存在しない場合，$\boldsymbol{x_{wp}}$ を弱パレート解と呼ぶ．定義上，パレート解は弱パレート解に含まれる．

3. 2 目的最短路問題

2 目的最短路問題は離散多目的最適化問題の一つで，単目的最短路問題を 2 目的に拡張させた問題である．本章では，2 目的最短路問題の定式化を行い，2 目的最短路問題における閉路の定義や性質について述べる．

3.1. 問題の定義と定式化

有向辺 $(u, v) \in A$ における 2 種類のコストを $\boldsymbol{c}_{uv} = (c^1_{uv}, c^2_{uv})$ と定義する．また，パス P のコストを $\boldsymbol{c}(P) = \sum_{(u,v)\in P} \boldsymbol{c}_{uv}$ と定義する．[6, 7, 8, 9, 11] などで扱われている従来の 2 目的最短路問題は，以下のように定義される．

入力：有向グラフ $G = (V, A)$，辺コスト (非負) $c^1 : A \to \mathbb{R}_{\geq 0}, c^2 : A \to \mathbb{R}_{\geq 0}$

出力：始点 s から終点 t までのパス P_{st} のうち 2 種のコスト $\boldsymbol{c}(P_{st})$ に関するパレート解

一方で，本研究で扱う 2 目的最短路問題は「負のコストを持つ辺や負閉路が含まれる」ような一般的な入力を許し，「弱パレート解」の列挙を目標とする．単目的最短路問題では，負閉路が存在すると，その閉路を繰り返し周回して到達できるノードまでのパス長をいくらでも小さくすることが可能である．2 目的最短路問題においても，閉路の存在によって同様の問題が生じ得るため，コストに応じた以下のような閉路を定義する．

定義 3.1 (正閉路・非正閉路・負閉路). 有向閉路 C が

$$c^1(C) > 0 \quad \text{かつ} \quad c^2(C) > 0$$

を満たすとき，C を正閉路と呼ぶ．また，正閉路でない閉路，すなわち

$$c^1(C) \leq 0 \quad \text{または} \quad c^2(C) \leq 0$$

を満たす閉路を非正閉路と呼び，

$$c^1(C) < 0 \quad \text{または} \quad c^2(C) < 0$$

を満たす閉路を負閉路と呼ぶ．

非正閉路・負閉路の具体例を図 1 に示す．図 1 (i) における (v_1, v_2, v_3, v_1) は非正閉路かつ負閉路であり，2 種の目的関数値のうち一方が負になっている．このとき，同じ閉路を何度も周回するパス（非単純路）を構築することで，一方の目的関数値をいくらでも小さくしつつ，他方の目的関数値を増加させるパスが無限に生成される．閉路を周回するたびに生成される新しいパスは，一方が減少するため既存のパスに強支配されず，結果として周回数に応じた無限個の弱パレート解が生成されてしまう．一方で，図 1 (ii) における (v_1, v_2, v_3, v_1) は負閉路ではないが非正閉路の例である．この場合は，閉路を周回するたびに一方のコストは変化せず，もう一方のコストのみが増加するパスが生成され，これらも互いに強支配されないため弱パレート解が有限個ではなくなってしまう例が存在する．

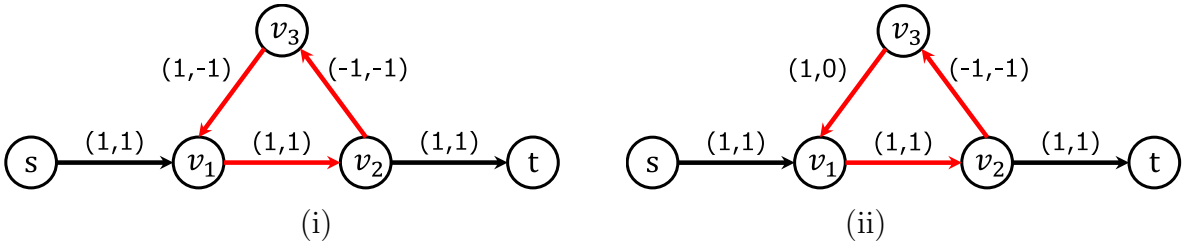


図 1: 非正閉路の具体例

このように閉路を繰り返し周回する非単純パスは，交通ネットワークなどの実問題において意味を持たないことが多い．そのような場合でも価値のある解集合の情報を得るため，本研究では

実行可能集合を有向グラフ G 上の単純パス集合

とする．本研究で扱うこの一般化された 2 目的最短路問題は，以下の通り表現できる．

定義 3.2 (2 目的最短路問題 **BSP**(s,t))**.**

入力：有向グラフ $G=(V,A)$，辺コスト $c^1 : A \to \mathbb{R}, c^2 : A \to \mathbb{R}$

出力：始点 s から終点 t までの単純パス集合上のコスト $\boldsymbol{c}$ に関するすべての弱パレート解

BSP(s,t) における (弱) パレート解のことを本論文では (弱) パレートパスとよぶ．なお，本論文では，コストベクトルが一致するパスが複数存在する場合であっても異なるパスとして扱い，それぞれを列挙対象とする．有限グラフ上の単純パスの総数は有限であるため，BSP(s,t) の実行可能解集合も有限である．したがって，s から t への単純パスが少なくとも 1 本存在する場合，BSP(s,t) の弱パレートパスは少なくとも 1 本存在し，その総数は有限である．

図 2 を用いて BSP(s,t) の弱パレートパスの具体例を説明する．なお, 本例は提案手法である WP-LC の動作例の説明において再度用いる．左図はノード集合 $V=\{s,v_1,v_2,v_3,v_4,t\}$ と各辺に 2 つのコスト (ベクトル値) が与えられた入力グラフである．このグラフにおいて始点 s から終点 t へのパスを探索した結果が右図である．右図において, 黄色の矢印で示されたパス① ($s \to v_3 \to v_2 \to t$) の合計コストを計算すると, $(1,4)+(1,-1)+(1,1)=(3,4)$ となる．同様に, 緑色の矢印のパス② ($s \to v_1 \to v_2 \to t$) の合計コストは $(1,5)+(-4,-1)+(1,1)=(-2,5)$ となる．これら 2 つのパスは, 互いにトレードオフの関係にあり, かつ他のどのパスにも支配されていないため, パレートパスとなる．一方, 青色の矢印で示されたパス③ ($s \to v_3 \to v_4 \to t$) の合計コストは $(1,4)+(1,0)+(-4,5)=(-2,9)$ となる．このパスは, パス②のコスト $(-2,5)$ と比較すると，第 1 目的関数の値は等しいが第 2 目的関数の値が劣っている ($9>5$)．したがって，パス③はパス②に支配されているためパレートパスではないが，強支配はされていないため，弱パレートパスとなる．赤色の矢印のパス④ ($s \to v_3 \to v_4 \to v_2 \to t$) の合計コストは $(1,4)+(1,0)+(0,0)+(1,1)=(3,5)$ となる．このパスは，パス①やパス③のコストと比較すると，強支配されないので，弱パレートパスとなる．

3.2. 非正閉路が存在しない場合の弱パレートパスの性質

前節では，非正閉路を含む一般の有向グラフを対象とするため，BSP(s,t) の実行可能解を単純パスに限定した．非正閉路が存在しない場合，BSP(s,t) のパレートパスおよび弱パレートパスは，単目的最短路問題における最短路と類似した部分路最適性を持つ．

命題 3.1. 所与のグラフ G に非正閉路が存在しないとき，s から t までのパレートパス P_{st} について，ノード u から v までの部分路 P_{uv} は BSP(u,v) のパレートパスである．

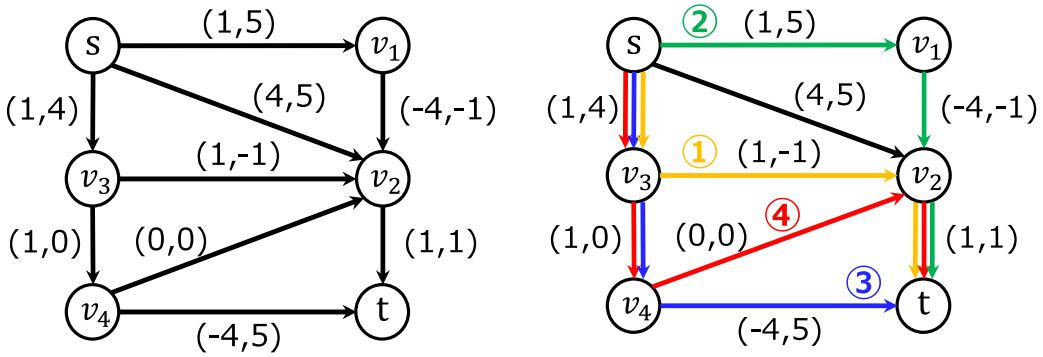


図 2: BSP 問題の具体例（左）とその解（右）

命題 3.2. 所与のグラフ G に非正閉路が存在しないとき，s から t までの弱パレートパス P_{st} について，ノード u から v までの部分路 P_{uv} は BSP(u, v) の弱パレートパスである．

証明. 命題 3.1 は命題 3.2 とほぼ同じ流れで示せるため，命題 3.2 の証明のみを示す．あるノード s から t までの弱パレートパス P_{st} について，P_{st} は u と v を通るとし，u から v までの P_{st} の部分路を P_{uv} とする．このとき，P_{uv} が弱パレートパスではないとすると，P_{uv} を強支配する単純 u-v パス P'_{uv} が存在する．つまり，P_{uv} と P'_{uv} のコストの関係は $\boldsymbol{c}(P'_{uv}) < \boldsymbol{c}(P_{uv})$ となる．

ここで

$$P' = P_{su} + P'_{uv} + P_{vt}$$

を考える．P' は一般には単純パスとは限らないため，閉路が含まれる一般の場合を含めて考える必要がある．P' が単純パスである場合，$\widetilde{P} = P'$ とおく．P' が単純パスでない場合，P' は少なくとも 1 つの閉路を含む．G に非正閉路が存在しないという仮定から，P' に含まれる閉路 C を除去すると，2 種の目的関数値はともに減少する．この操作を P' から閉路がなくなるまで繰り返して得た単純 s-t パスを $\widetilde{P}$ とおく．P' が単純パスである場合とそうではない場合，どちらにおいても

$$\boldsymbol{c}(\widetilde{P}) \leq \boldsymbol{c}(P') < \boldsymbol{c}(P_{st})$$

が成り立ち，$\widetilde{P}$ は P_{st} を強支配する．これは P_{st} が弱パレートパスであることに矛盾する．よって，所与のグラフ G に非正閉路が存在しないとき，弱パレートパスの部分路は弱パレートパスである． □

注意 3.1. 命題 3.2 は非正閉路が存在する一般の BSP(u, v) では成り立たない．図 3 について，(s, v_3, v_1) はコストが $(0, 0)$ の弱パレートパスだが，コストが $(0, 0)$ の部分パス (s, v_3) は弱パレートパスではない．なぜなら，(s, v_3) はコスト $(-1, -1)$ のパス (s, v_1, v_3) に強支配されるからである．

命題 3.2 の性質は，本研究で用いるラベル削除処理と密接に関係する．非正閉路が存在しない場合には，弱パレートパスの部分路も弱パレートパスであるため，単純に強支配されるラベルを削除するだけで，弱パレートパスの全列挙が保証される．一方で，非正閉路が存在する場合には上記の部分路最適性が成立しないため，強支配関係のみを用いたラベル削除では一部の弱パレートパスを列挙できない可能性がある．この問題に対処するため，提案アル

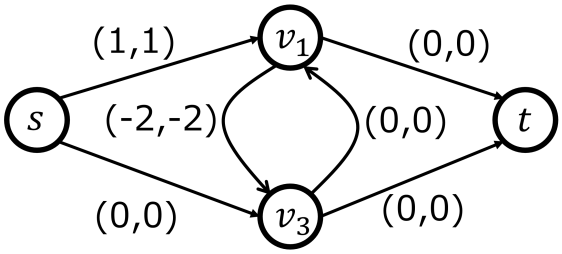


図 3: BSP(s,t) に関して命題 3.2 が成立しない例

ゴリズムでは 4.2.4 節で述べるラベル削除機構を導入する．さらに，提案アルゴリズムは非正閉路を検知し，図 1 (i) のように負重みをもつ閉路か，(ii) のように 0 重みをもつ閉路かを分類する機能を有している．特に，少なくとも一方の目的の閉路コストが非正となる非正閉路については，そのパスおよびコストを記録する．この閉路検知機構の詳細については 4.2.5 節で説明する．

4. WP-LC

本節では，弱パレートパスを全列挙するアルゴリズムである WP-LC (Algorithm 1) を提案する．WP-LC はラベル修正法にあたる．このアルゴリズムは負の重みを持つ辺や非正閉路を含む，より一般的なグラフで動作する．

Algorithm 1 は WP-LC の疑似コードである．このアルゴリズムは，大きく分けて事前探索と本探索の 2 つから構成される．以下に各処理のステップを解説する．なお，本アルゴリズムではノード node のラベル l を以下のように定義する．

$$l = (\mathrm{node}, \mathbf{cost} = (\mathrm{cost}^1, \mathrm{cost}^2), \mathrm{parent})$$

ここで，parent はラベル l の親ラベルを表し，parent を辿ることで，s から node までのパスを得ることができる．そして，**cost** は頂点 s から node までのこのパス上のコストの総和を表す．また，ラベルの支配・強支配関係を以下のように定義する．

定義 4.1. ラベル l, l' ($l.\mathrm{node} = l'.\mathrm{node}$) に対して，$l.\mathbf{cost} \leq l'.\mathbf{cost}$ かつ $l.\mathbf{cost} \neq l'.\mathbf{cost}$ ならば，l が l' を支配する ($l \prec l'$) といい，$l.\mathbf{cost} < l'.\mathbf{cost}$ ならば，l が l' を強支配する ($l \prec_{str} l'$) という．

4.1. 事前探索

まず，2 目的の本探索に先立ち，Forward PreSearch と Backward PreSearch という事前探索を行う．どちらの探索も各目的を独立に扱う単目的の探索に基づく．

4.1.1. Forward PreSearch

始点 s から到達可能な有向閉路のうち，$c^1(C_1) < 0$ を満たす閉路 C_1 を考える．このような閉路 C_1 から到達可能なノードの集合を D_1 と定義する．同様に，始点 s から到達可能かつ $c^2(C_2) < 0$ を満たす閉路 C_2 から到達可能なノードの集合を D_2 と定義する．さらに

$$V_{\mathrm{div}} = D_1 \cup D_2$$

を**発散ノード集合**と定義し，$v \in V_{\mathrm{div}}$ を**発散ノード**と呼ぶ．この定義から

$$u \in V_{\mathrm{div}} \text{ かつ } u \text{ から } v \text{ に到達可能} \Longrightarrow v \in V_{\mathrm{div}} \tag{4.1}$$

という性質が成り立つ．さらに，各ノード v について

$$\mathrm{divergent_flags}[v] = \begin{cases} \mathrm{true} & (v \in V_{\mathrm{div}}), \\ \mathrm{false} & (v \notin V_{\mathrm{div}}) \end{cases}$$

と真偽値配列を定義する．

Forward PreSearch では，第 1 目的と第 2 目的それぞれに関して，始点 s から Bellman–Ford 法などにより順方向の探索を行う．探索により，第 1 目的 (第 2 目的) 関数値が負の無限大へと発散するノードの集合 $D_1(D_2)$ を求め，各ノード v に対して真偽値配列 $\mathrm{divergent_flags}[v]$ として記録する．発散ノードの特定は，後の探索における不要な枝刈りを防ぎ，閉路を適切に処理するために不可欠である．

4.1.2. Backward PreSearch

辺数が高々 $n-1$ の v-t パスの集合を $\mathcal{P}_{vt}^{\leq n-1}$ とする．各ノード v に対し，d_v を

$$\boldsymbol{d}_v = \left(\min_{P \in \mathcal{P}_{vt}^{\leq n-1}} c^1(P), \min_{P \in \mathcal{P}_{vt}^{\leq n-1}} c^2(P) \right) \tag{4.2}$$

と定義する．ただし，$\mathcal{P}_{vt}^{\leq n-1}$ が空集合の場合は $\boldsymbol{d}_v = (+\infty, +\infty)$ とする．このように定義すると，任意の単純 v-t パスは辺数が高々 $n-1$ であるため，$\boldsymbol{d}_v$ は任意の単純 v-t パスのコストの下界を与える．

下界の計算は，各有向辺の向きを逆にした逆グラフ上で，終点 t を始点として，各目的を独立したコストとして行う．両目的に関して負閉路が存在しないことが既知の場合は，各目的について SPFA [10, 14] を実行する．この場合，v から t に到達可能ならば，各目的について最短 v-t パスとして単純パスを選ぶことができる．その辺数は高々 $n-1$ であるため，SPFA で得られる最短距離は (4.2) の値に一致する．

一方で，負閉路が存在する場合は，各目的について Bellman–Ford 法を実行し，$n-1$ 回の反復後の値を $\boldsymbol{d}_v$ とする．Bellman–Ford 法の各反復では，前回の反復終了時の距離ラベルのみを参照してラベルの更新を行うことで，$n-1$ 回の反復後に (4.2) の値が求められる．

上界の構成には，第 1 目的および第 2 目的の探索で得られた 2 つの単純 s-t パスを利用する．SPFA を用いる場合は，第 1 目的に関する最短単純 s-t パスを P_1，第 2 目的に関する最短単純 s-t パスを P_2 とする．Bellman–Ford 法を用いる場合は，辺数最小の単純 s-t パスの集合において，第 1 目的を最小化するパスを P_1，第 2 目的を最小化するパスを P_2 とする．これらは，各辺の向きを逆にしたグラフ上で，各目的について Bellman-–Ford 法を実行して求める．各頂点 v について，始点 t から v までの距離が初めて有限となる反復の終了時に得られるラベルのうち，各目的のコストを最小化するラベルを保存しておくことで求められる．そして，枝刈りに用いる上界を

$$\boldsymbol{w} = \left(c^1(P_2), c^2(P_1) \right) \tag{4.3}$$

と定める．s-t パスが存在する場合，いずれの方法でも P_1 と P_2 は同一のパス集合上で各目的を最小化するため，$c^1(P_1) \leq c^1(P_2)$ および $c^2(P_2) \leq c^2(P_1)$ が成り立つ．したがって，

$$\boldsymbol{c}(P_1) \leq \boldsymbol{w} \text{ かつ } \boldsymbol{c}(P_2) \leq \boldsymbol{w} \tag{4.4}$$

であり，$\boldsymbol{w}$ は P_1, P_2 のコストの上界となる．s-t パスが存在しない場合は，$\boldsymbol{w} = (+\infty, +\infty)$ とする．本節で計算された下界 $\boldsymbol{d}_v$ と上界 $\boldsymbol{w}$ は，本探索において，終点に到達しても弱パレートパスになり得ないラベルを早期に特定し，探索空間を削減する枝刈りのための基準として利用される．

4.2. 本探索

まず，探索状態を管理する主要なデータ構造を定義する．各ノード $v \in V$ に対して $L[v]$ は，これまでに発見された s-v 単純パスに対応するラベルのうち，ラベルの比較・更新のために保持されているものの集合である．$L[v]$ 内のラベルは，第 1 目的コストを第 1 キー，第 2 目的コストを第 2 キーとするコストベクトル $(\mathrm{cost}_1, \mathrm{cost}_2)$ の辞書式順序 (昇順) で保存される．一方，$U[v]$ はノード v で受理されたラベルのうち，まだ伝播処理に取り出されてい

Algorithm 1 WP-LC

Require: グラフ $G = (V, A)$，始点 s，終点 t

1: divergent_flags ← Forward PreSearch ▷ 事前探索
2: $\boldsymbol{d}_v$ $(v \in V), \boldsymbol{w}$ ← Backward PreSearch
3: $l_{\mathrm{start}} = (s, 0, 0, \mathrm{null})$, $L[v] = \emptyset$, $U[v] = \emptyset$ $(v \in V)$
4: $Q \leftarrow \{s\}$, $L[s] \leftarrow \{l_{\mathrm{start}}\}$, $U[s] \leftarrow \{l_{\mathrm{start}}\}$ ▷ 本探索
5: **while** $Q \neq \emptyset$ **do**
6: $u \leftarrow Q$.pop_front() ▷ $u, v \in V$
7: Labels ← $U[u]$, $U[u] \leftarrow \emptyset$
8: **for** arc ∈ adj[u] **do** ▷ adj[u] はノード u から出る有向辺の集合
9: $v \leftarrow$ arc.to ▷ arc を通るノード u の後続ノード
10: change_flag ← false ▷ $L[v]$ の更新フラグ
11: **for** $l \in$ Labels **do**
12: **new_cost** ← l.**cost** + $\boldsymbol{c}_{\mathrm{arc}}$
13: **if** IsPruned(v, **new_cost**, $L[t]$) **then continue** ▷ 枝刈り
14: **end if**
15: **if** CheckAndRecordCycle($l, v,$ arc) **then continue**
16: **end if**
17: $l_{\mathrm{new}} \leftarrow$ UpdateWeakParetoSet(v, **new_cost**, l) ▷ $L[v]$ の更新
18: **if** $l_{\mathrm{new}} \neq$ null **then**
19: $U[v]$.push_back(l_{new})
20: change_flag ← true
21: **end if**
22: **end for**
23: **if** change_flag $\wedge$ $v \notin Q$ **then**
24: InsertQueueSLF(v) ▷ 拡張 SLF を用いたキューへの挿入
25: **end if**
26: **end for**
27: **end while**
28: **return** $L[t]$ ▷ 終点 t に関する弱パレートラベルリストを出力

ないものの集合である．既存ラベルの削除は $L[v]$ からの除去のみを意味し，$U[v]$ からは除去しない．また，Q は未伝播ラベルを持つ処理待ちノードを管理するキューである．また，本論文における**緩和**とは，未伝播のラベルを後続ノードへ伝播し，枝刈りや閉路検知，強支配関係の判定を経て，新たな保持ラベルとして追加するか否かを判定する Algorithm 1 の 5 行目から 28 行目までの一連の処理を指す．

本探索では，$Q = \emptyset$ になるまでこのラベルの緩和処理を繰り返す．各反復において，まず Q から先頭のノード u を取り出す (pop_front)．このとき，ノード u から後続ノードへ伝播させるために，未伝播ラベル集合 $U[u]$ を一時変数 Labels に移し，$U[u]$ を空にする．

次に，ノード u から出る各有向辺 arc（後続ノードを v とする）について，ノード v でのラベル更新の有無を記録するフラグ change_flag を初期化した後，Labels 内の各ラベル l に対して以下の処理を行う．まず，ラベル l のコストベクトル $l.\mathbf{cost}$ に辺 arc のコストベクトル $\boldsymbol{c}_{\text{arc}}$ を加算し，ノード v への新たな到達コスト **new_cost** を計算する．この **new_cost** に対し，関数 IsPruned を用いて枝刈り判定を実施する．Duque ら [5] や Sedeño-Noda と Colebrook [13] に倣い，事前探索で得られた下界や上界，及びすでに終点に到達している解候補を基準とし，この先の緩和で弱パレートパスになり得ないと判断された場合は，直ちに伝播を打ち切る．枝刈りに関しては 4.2.6 節で詳述する．

枝刈りされなかった各ラベルについて，次に関数 CheckAndRecordCycle を実行し，辺 arc の追加によってパスに同一ノードが再出現するかを確認する．閉路が検知された場合には，当該ラベルの伝播を中止する．ただし，後で示す注意 4.1 の通り，入力グラフに非正閉路が存在しないことが既知の場合には，本処理を省略できる．

閉路検知も通過したラベルについては，関数 UpdateWeakParetoSet によって，既存ラベルとの強支配関係およびパス履歴に基づく削除可否が判定され，ラベル集合 $L[v]$ の更新が行われる．新たな候補ラベルが CanDominate によって安全に削除可能であると判定されなかった場合は，l_{new} が生成されて $L[v]$ に追加され，次の伝播のために未伝播ラベル集合 $U[v]$ にも追加される．このとき，change_flag フラグを true にする．ノード u の持つすべての未伝播ラベルの検証を終えた後，change_flag が true であり，かつノード v が現在キュー Q に存在しない場合，Q へノード v を挿入する．この際，単なる FIFO (First In, First Out) ではなく，[6] における多目的ラベル修正法で有効性が示されている SLF をベースとし，さらに推定費用ベクトルを用いた拡張を行った挿入戦略を適用している．

以上の処理を $Q = \emptyset$ になるまで繰り返し，得られた $L[t]$ を出力として返す．なお，本アルゴリズムの核となる枝刈り（IsPruned），閉路検知（CheckAndRecordCycle），および履歴の包含関係チェックを伴うラベル更新関数（UpdateWeakParetoSet）の詳細については，以降の項で詳述する．

4.2.1. アルゴリズムの基本動作の具体例

WP-LC の全体像を把握するため，以下の図 4(i) から図 4(iv) を用いて，基本的なラベル修正とキューを用いた伝播の過程を説明する．この具体例で用いるグラフは，図 2 と同じものを使用する．なお，本節ではアルゴリズムの基本的な動作の理解を目的とするため，枝刈りや閉路検知は機能しないものとし，キューは単純な FIFO のデータ構造として動作するものとして説明する．各図において，各ノードのラベルは黒枠内にかき，赤字で示されたラベルは新しく生成されたラベルである．また，赤字のノードは現在キューに入っているノードを，枠線が赤色で強調されたノードは，そのステップにおいてキューから取り出され，後続ノードへの伝播処理を行っている状態を表す．

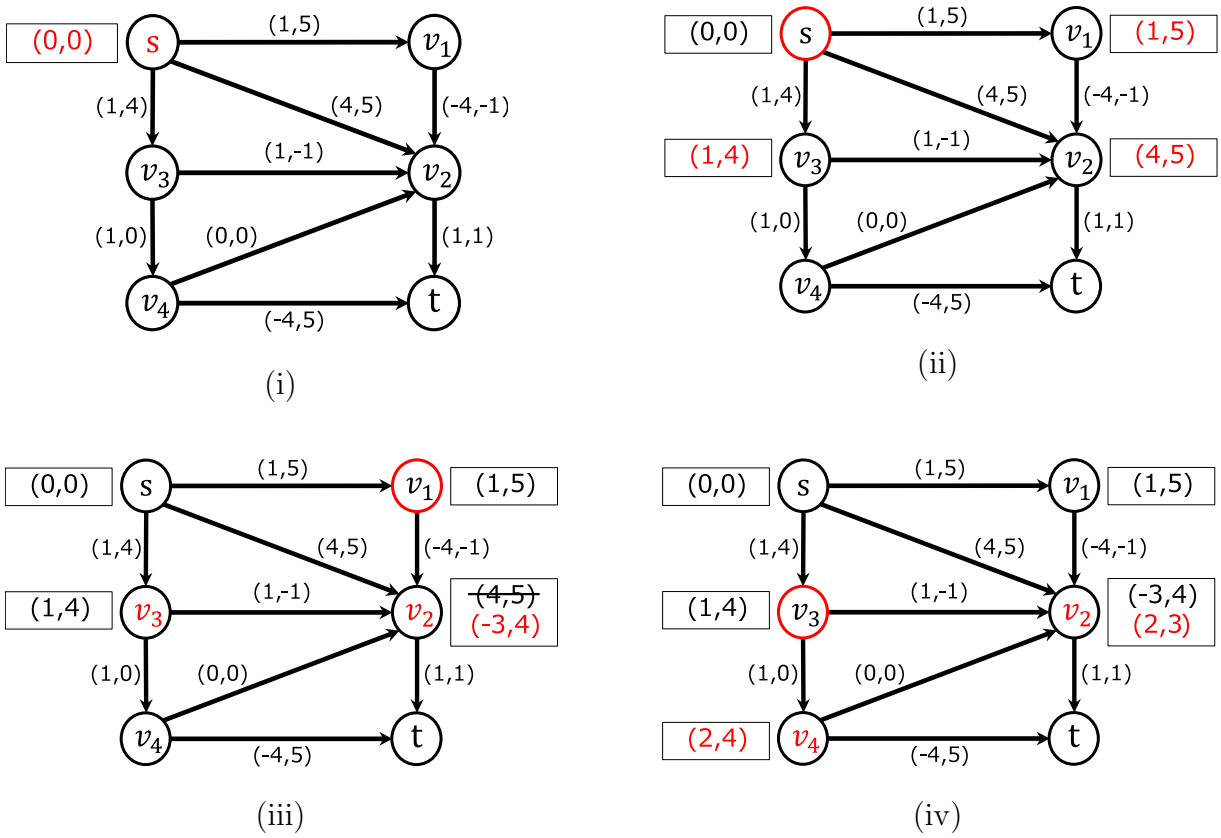


図 4: アルゴリズムの動作例

- **図 4(i)（初期状態）**：探索の開始時，始点 s に初期ラベル $(0,0)$ が生成され，未伝播ノードを管理するキュー Q に s が挿入される．他のすべてのノードのラベル集合は空である．
- **図 4(ii)（s の伝播）**：Q から s を取り出し，s の後続ノード v_1, v_3, v_2 へラベルを伝播させる．それぞれの有向辺のコストが加算され，v_1 に $(1,5)$，v_3 に $(1,4)$，v_2 に $(4,5)$ という新たなラベルが生成される．これらはすべて各ノードの保持ラベルとして登録され，ラベルが更新された v_1, v_3, v_2 の順で Q に挿入される．
- **図 4(iii)（v_1 の伝播と強支配による更新）**：Q の先頭から v_1 を取り出し，保持しているラベル $(1,5)$ を後続ノードへ伝播する．v_2 への辺コストが $(-4,-1)$ であるため，新たな到達コストは $(1-4, 5-1) = (-3,4)$ と計算される．ここで v_2 の既存ラベル $(4,5)$ と比較を行う．新ラベル $(-3,4)$ は，両方の目的関数値において既存ラベル $(4,5)$ を下回るため，これを強支配する．その結果，不要となった $(4,5)$ は v_2 の保持ラベル集合 $L[v_2]$ から削除され，代わりに $(-3,4)$ が保持される．
- **図 4(iv)（v_3 の伝播と複数ラベルの保持）**：続いて Q から v_3 を取り出し（v_3 の赤枠），ラベル $(1,4)$ を伝播する．v_4 へは $(1,0)$ の辺を通じて $(2,4)$ が新たに生成され，Q に v_4 を追加する．一方，v_2 へは $(1,-1)$ の辺を通じて到達コスト $(2,3)$ が計算される．ここで v_2 が現在保持しているラベル $(-3,4)$ と比較を行うと，第 1 目的は $-3 < 2$ で既存ラベルが優れているが，第 2 目的は $4 > 3$ で新ラベルが優れている．両者は互いに強支配関係を持たないため，いずれも削除されず，保持ラベルとして v_2 のリスト $L[v_2]$ に並存して保持される．このとき，Q には v_2, v_4 の順で挿入されている．

Algorithm 2 InsertQueueSLF

function INSERTQUEUESLF(v)
  if $Q \neq \emptyset$ **then** ▷ 各目的の最小推定費用ベクトルを用いた拡張 SLF の適用
    $q_{\text{front}} \leftarrow Q.\text{front}()$ ▷ Q の先頭ノード
    $l_{\text{front}} \leftarrow L[q_{\text{front}}].\text{front}()$ ▷ $L[q_{\text{front}}]$ の先頭ラベル
    $est^1 \leftarrow \min_{l \in U[v]} (l.\text{cost}^1 + d_v^1)$
    $est^2 \leftarrow \min_{l \in U[v]} (l.\text{cost}^2 + d_v^2)$
    if $est^1 < l_{\text{front}}.\text{cost}^1 + d_{q_{\text{front}}}^1 \wedge est^2 < l_{\text{front}}.\text{cost}^2 + d_{q_{\text{front}}}^2$ **then**
      Q.push_front(v) ▷ 両目的で先頭ノードを下回る場合は先頭へ
    else
      Q.push_back(v) ▷ それ以外は末尾へ挿入
    end if
  else
    Q.push_back(v)
  end if
end function

このように，キューを用いてノードからノードへラベルを伝播させながら，各ノードにおいて強支配関係およびパス履歴に基づいて安全に削除可能なラベルを除去し，以降の探索に必要なラベルを保持するのが，本アルゴリズムの基本的な動作である．この一連の処理を繰り返し行うことで，s-t 間の弱パレートパスを全列挙する．

4.2.2. InsertQueueSLF

関数 InsertQueueSLF では，探索空間を削減するため，Guerriero と Musmanno [6] によって導入された SLF (Small Label First) に対し，推定費用ベクトルを用いた拡張を行っている．従来の SLF は，キューへの挿入候補となるノードとキュー先頭のノード間で，始点からの現在の到達コストのみを比較して挿入位置（先頭か末尾か）を決定する．しかし，この手法は終点までの距離を考慮せず，序盤のコストが小さければ，そのラベルについて優先的に探索するため，無駄な探索を行いうる．これに対し，WP-LC では，関数 ISPRUNED の計算でも用いる終点到達時の予測総コストの下界である推定費用ベクトル $\boldsymbol{est} = l.\mathbf{cost} + \boldsymbol{d}_v$ を SLF の比較基準として採用している．

Algorithm 2 に具体的な挿入処理の手順を示す．ここではノード v をキュー Q に挿入する際，$U[v]$ に含まれる未伝播ラベルから計算される推定費用ベクトルと，現在の Q の先頭ノード q_{front} における辞書式順序で先頭に位置するラベル l_{front} の推定費用ベクトルを直接比較する．そして，ノード v の推定費用ベクトルが，代表ラベル l_{front} の推定費用ベクトルよりも完全に小さい場合のみ，v をキューの先頭に挿入し（push_front），そうでなければ末尾に挿入する（push_back）．この拡張により，単に現在コストが小さいノードではなく，最終的に終点 t で弱パレートパスになり得る見込みが高いノードが優先的にグラフ全体へ伝播される．

4.2.3. UpdateWeakParetoSet

関数 UpdateWeakParetoSet (Algorithm 3) は，引数ノード v におけるラベル集合 $L[v]$ の更新を行う．まず，引数から新たな候補ラベル l_{new} を一時生成する．このとき，候補ラベル集合

Algorithm 3 UpdateWeakParetoSet

1: **function** UpdateWeakParetoSet(v, **new_cost**, l_{parent})
2: ▷ 候補となる新ラベルを定義（まだリストには追加しない）
3: $l_{\text{new}} = (\text{node} = v, \textbf{cost} = \textbf{new_cost}, \text{parent} = l_{\text{parent}})$
4: **for** $l \in L[v]$ **do**
5: **if** $l.\textbf{cost} = l_{\text{new}}.\textbf{cost} \wedge l.\text{parent} = l_{\text{new}}.\text{parent}$ **then**
6: **return** null ▷ 全く同じラベルは削除
7: **end if**
8: **end for**
9: **for** $l \in \{l' \in L[v] \mid l'.\text{cost}^1 < l_{\text{new}}.\text{cost}^1\}$ **do** ▷ 既存ラベルに強支配されるか確認
10: **if** CanDominate(l, l_{new}, v) **then** ▷ 包含関係のチェック
11: **return** null ▷ 強支配されるため l_{new} を削除
12: **end if**
13: **end for**
14: **for** $l \in \{l' \in L[v] \mid l_{\text{new}}.\text{cost}^1 < l'.\text{cost}^1\}$ **do** ▷ 既存ラベルを強支配できるか確認
15: **if** CanDominate(l_{new}, l, v) **then**
16: $L[v] \leftarrow L[v] \setminus \{l\}$ ▷ 強支配できるため既存の l を削除
17: **end if**
18: **end for**
19: $L[v] \leftarrow L[v] \cup \{l_{\text{new}}\}$ ▷ 保持ラベル集合の更新
20: **return** l_{new} ▷ 追加された新しいラベルを返す
21: **end function**

$L[v]$ 内に l_{new} とコストと親ラベルが共に一致するようなラベルがあれば，パスの重複とみなして，l_{new} を直ちに削除する．次に，新ラベルと既存ラベルとの強支配関係，および強支配されたラベルを安全に削除できるかを判定する．ここでは第 1 目的のコストに基づいて比較対象を絞り込む．$l'.\text{cost}^1 < l_{\text{new}}.\text{cost}^1$ のとき，l_{new} は既存ラベル l' に強支配されることがあっても，強支配することはない．そのため，l' と l_{new} に対して関数 CanDominate（詳細は 4.2.4 節および命題 4.3 を参照）を適用し，$l' \prec_{\text{str}} l_{\text{new}}$ が成り立ち，かつ CanDominate(l', l_{new}, v) が真を返す場合，l_{new} は弱パレートパスの生成に必要がないことが保証されるため，削除される．反対に，$l_{\text{new}}.\text{cost}^1 < l'.\text{cost}^1$ のとき，l_{new} は l' を強支配することはあっても，l' に強支配されることはない．そのため，$l_{\text{new}} \prec_{\text{str}} l'$ が成り立ち，かつ CanDominate(l_{new}, l', v) が真を返す場合，既存ラベル l' は保持ラベルとして $L[v]$ に残す必要がないことが保証されるため，$L[v]$ から削除される．これらの判定によって削除されなかった l_{new} は，ノード v における保持ラベルとして $L[v]$ に追加され，以降の伝播処理のために出力される．なお，9 行目および 14 行目における比較対象の抽出は，実装上は第 1 目的コストを第 1 キー，第 2 目的コストを第 2 キーとする辞書式順序で保存されたラベル配列に対して，二分探索を行うことで高速に処理できる．

4.2.4. CanDominate

関数 CanDominate (Algorithm 4) の引数ラベル l_{win}, l_{lose} は，$l_{\text{win}}.\text{cost}^1 \leq l_{\text{lose}}.\text{cost}^1$ であることを前提とし，l_{win}, l_{lose} をそれぞれ勝者ラベル，敗者ラベルと呼ぶことにする．また引数ノード v は評価中のノードであり，$l_{\text{win}}.\text{node} = l_{\text{lose}}.\text{node} = v$ である．注意 3.1 でも示

Algorithm 4 CanDominate

```
1: function CanDominate(l_win, l_lose, v)
2:     if l_win ≺_str l_lose then
3:         if divergent_flags[v] = false ∨ v = t then
4:             return true                    ▷ 非発散ノード，終点では常に l_lose の削除を許可
5:         end if
6:         H_win ← {u ∈ Path(l_win) | divergent_flags[u] = true}
7:                                            ▷ l_win のパス上の発散ノード集合
8:         H_lose ← {u ∈ Path(l_lose) | divergent_flags[u] = true}
9:                                            ▷ l_lose のパス上の発散ノード集合
10:        if H_win ⊆ H_lose then
11:            return true                    ▷ 包含関係を満たせば l_lose の削除を許可
12:        else
13:            return false
14:        end if
15:    else
16:        return false
17:    end if
18: end function
```

した通り，現時点で勝者ラベル l_{win} が敗者ラベル l_{lose} を強支配していたとしても，両ラベルの訪問済みノード集合が異なる可能性があるため，単純パス制約の下で今後利用可能な後続パスの集合が一致するとは限らない．したがって，強支配関係のみを根拠として敗者ラベル l_{lose} を削除することはできない．本関数は，この敗者ラベルを安全に削除できるかを判定するために関数 UpdateWeakParetoSet によって呼び出される．

判定は将来訪問可能な発散ノード集合の包含関係に基づく．まず，引数ノード v が発散ノードでない場合，(4.1) より，始点 s から v に至る両ラベルのパス上には発散ノードが存在しない．したがって

$$H_{\text{win}} = H_{\text{lose}} = \emptyset$$

であり，Algorithm 4 における 10 行目の包含関係

$$H_{\text{win}} \subseteq H_{\text{lose}}$$

は自動的に満たされるため，強支配されたラベルの削除を許可する．また，$v = t$ の場合には後続ノードへの拡張を考える必要がないため，強支配されたラベルの削除を許可する．

一方，ノード v が発散ノードの場合は，l_{win} と l_{lose} が通ったそれぞれの発散ノード集合の包含関係を確認する．Algorithm 4 の $\textsc{Path}(l)$ はラベル l のパスのノード集合である．ここで

$$H_{\text{win}} = \textsc{Path}(l_{\text{win}}) \cap V_{\text{div}},\ H_{\text{lose}} = \textsc{Path}(l_{\text{lose}}) \cap V_{\text{div}} \tag{4.5}$$

と定義し，包含関係 $H_{\text{win}} \subseteq H_{\text{lose}}$ を判定し，真ならば l_{lose} を削除することを許可する．

図 5 を用いて，この判定の具体的な動作を説明する．図中の破線で示されたノード v_i, v_j は発散ノードである．今，ノード v_i において 2 つのラベルが競合した状況を考える．一つ

は始点 s から直接 v_i に到達した赤色のラベル l_{win} であり，もう一つは v_j を経由して v_i に到達した青色のラベル l_{lose} である．ここで，$l_{\text{win}} \prec_{\text{str}} l_{\text{lose}}$ と仮定する．このとき，それぞれのパスがこれまでに訪問した発散ノードの集合は，$H_{\text{win}} = \{v_i\}, H_{\text{lose}} = \{v_i, v_j\}$ となる．これらを比較すると，$H_{\text{win}} \subseteq H_{\text{lose}}$ が成り立つ．これは，l_{lose} がすでに v_j を訪問済みであり，これ以降の反復の緩和で再利用できない一方，l_{win} は未訪問の v_j をこれ以降の反復で利用できる可能性を保持していることを意味する．すなわち，l_{win} は現在のコストが優れているだけでなく，将来訪問可能な発散ノード集合においても l_{lose} を包含しているため，l_{lose} を安全に削除できる（true を返す）と判定される．なお，ここで false となり途中ノードで保持された冗長なラベルであっても，終点 t に到達した時点では削除が許可される（3 行目の条件 $v = t$）ため，最終的な出力集合 $L[t]$ には弱パレートパスのみが厳密に抽出される．

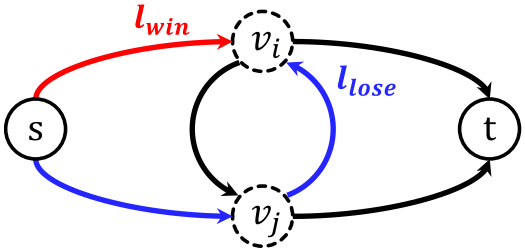


図 5: CanDominate の具体例

4.2.5. CheckAndRecordCycle

関数 CheckAndRecordCycle (Algorithm 5) は，生成候補パスが単純パス制約を満たすかを確認するとともに，検知された閉路の分類・記録を行う．まず，ノード v への緩和時，現在のラベル l の親ラベルを遡ってパスを確認する．遡ったパスに v がなければ閉路を持たないため，記録せずに処理を終える．遡ったパス内に v が存在して閉路を検知した場合，今回 v に到達する総コストと過去に訪問した際の総コストの差分をとることで，再探索なしに閉路 1 周分の純粋なコスト変動 $\Delta\mathbf{cost}$ を算出する．その後，$\Delta\mathbf{cost}$ に基づき閉路を分類し，非正閉路であれば，パス P と $\Delta\mathbf{cost}$ を記録する．なお，閉路が検知された場合，その拡張によって得られるパスは単純パスではないため， BSP(s, t) の実行可能解となり得ない．したがって，閉路のコストの種類にかかわらず true を返し，当該ラベルの伝播を直ちに打ち切る．この処理により，本探索で生成されるラベルは単純パスに対応するものに限定される．

注意 4.1. 入力グラフに非正閉路が存在しない場合，関数 CheckAndRecordCycle の実行を省略することができる．このような入力において，任意の閉路 C は正閉路であるから

$$c^1(C) > 0, \qquad c^2(C) > 0$$

を満たす．よって，あるラベルから閉路 C を一周して同じノードに戻って生成される新たなラベルは，閉路に入る直前のラベルに強支配される．また，非正閉路が存在しない場合は $V_{\text{div}} = \emptyset$ であるため，この生成された新たなラベルは UpdateWeakParetoSet において削除される．なお，閉路に入る直前のラベルが既に $L[v]$ から削除されている場合でも，$V_{\text{div}} = \emptyset$ の下では，そのラベルを強支配して削除した保持ラベルが存在するため，閉路を含む新たなラベルは同様に削除される．したがって，CheckAndRecordCycle を実行しなくても，WP-LC の停止性および弱パレートパスの列挙結果には影響しない．数値実験で

Algorithm 5 CheckAndRecordCycle

```
1: function CheckAndRecordCycle(l, v, arc)
2:     P ← [v]                                   ▷ 閉路（ノードの列）を格納する初期リスト
3:     l_cur ← l
4:     found ← false          ▷ l_cur の親を辿り，閉路の起点 v の探索とパスの復元を行う
5:     while l_cur ≠ null do
6:         P.push_front(l_cur.node)
7:         if l_cur.node = v then
8:             found ← true
9:             break
10:        end if
11:        l_cur ← l_cur.parent
12:    end while
13:    if not found then
14:        return false                          ▷ 閉路が存在しない場合は記録せずに終了
15:    end if
16:    Δcost ← (l.cost + c_arc) − l_cur.cost          ▷ 閉路 1 周分のコストを計算
17:    if Δcost_1 ≤ 0 ∨ Δcost_2 ≤ 0 then
18:        RecordCycle(P, Δcost)                      ▷ 非正閉路を記録
19:    end if
20:    return true                               ▷ 閉路を検知したため true を返す
21: end function
```

は，非正閉路が存在しないことが既知のインスタンスについては，計算時間短縮のため閉路検知処理を省略している．

4.2.6. IsPruned

関数 IsPruned (Algorithm 6) は，生成されたラベルが最終的な弱パレートパスになり得るかを事前評価し，不要な伝播を早期に打ち切る．まず，事前探索で得られたノード v から終点 t への下界 $\boldsymbol{d}_v$ と，s-t 間の上界 $\boldsymbol{w}$ を参照する．d_v のいずれかの成分が $+\infty$ の場合は，終点に到達不能として直ちに枝刈りする．到達可能な場合，現在の到達コスト **new_cost** に下界 $\boldsymbol{d}_v$ を加算し，終点到達時の予測総コストの下界である推定費用ベクトル $\boldsymbol{est}$ を算出する．$\boldsymbol{est}$ はそのパスのコストの下界でもある．$\boldsymbol{w} < \boldsymbol{est}$ の場合，式 (4.4) より事前探索で求めた単純パス P_1 について $\boldsymbol{c}(P_1) \leq \boldsymbol{w}$ であるため，候補を延長して得られる任意の単純 s-t パス P について，$\boldsymbol{c}(P_1) \leq \boldsymbol{w} < \boldsymbol{est} \leq \boldsymbol{c}(P)$ が成り立つ．したがって，P は P_1 に強支配されるため，枝刈りを行う．さらに，すでに終点 t に到達し保持されている弱パレートパス候補の集合 $L[t]$ と $\boldsymbol{est}$ を比較する．ここでも第 1 目的で比較対象を絞り込み，$\boldsymbol{est}$ が既存の終点ラベルに強支配される場合，その先の緩和は無駄なため枝刈り（true）する．すべての判定を通過した有望なラベルのみが枝刈りを免れる．

4.3. WP-LC の正当性

本節では，提案手法である WP-LC が，負辺や多様な閉路を含む任意のグラフ $G = (V, A)$ において必ず停止し，かつ弱パレートパスを一つも漏らすことなく列挙できることを証明す

Algorithm 6 IsPruned

```
1: function IsPruned(v, new_cost, L[t])
2:                                  ▷ d_v：式 (4.2) で定めた単純 v-t パスのコストの下界
3:                                  ▷ w：式 (4.3) で定めた枝刈り基準ベクトル
4:     if d_v^1 = ∞ ∨ d_v^2 = ∞ then
5:         return true                          ▷ 終点に到達不可能な場合は枝刈り
6:     end if
7:     est ← new_cost + d_v                     ▷ 推定費用ベクトルの算出
8:     if w^1 < est^1 ∧ w^2 < est^2 then
9:         return true                          ▷ 上界 w を超える場合は枝刈り
10:    end if
11:    for l ∈ {l' ∈ L[t] | l'.cost^1 ≤ est^1} do   ▷ 既存の弱パレートパス候補との比較
12:        if l.cost < est then
13:            return true                      ▷ 既存の解に強支配される場合は枝刈り
14:        end if
15:    end for
16:    return false
17: end function
```

る．WP-LC の正当性は，以下の 4 つの命題によって保証される．

命題 4.1. WP-LC は有限回の反復で必ず停止する．

証明. まず, 事前探索では, SPFA をグラフに負閉路が含まれない場合にのみ適用し, 負閉路が含まれる場合には有限回の緩和処理と到達可能性の探索を行うため, いずれの場合も有限時間で終了する．本探索では, 各反復においてラベル l のパス $P_l = (s = v_0, v_1, \ldots, v_i)$ を親ラベルに沿って遡り, 後続ノード v_j が $v_j \in P_l$ を満たすかを確認する（CheckAndRecordCycle）．もし $v_j \in P_l$ であれば，閉路 $C = (v_j, \ldots, v_i, v_j)$ が形成されるため直ちに伝播を打ち切る．

これにより，アルゴリズムが探索するパスの集合は，単純パスの集合 $\mathcal{P}_{\text{simple}}$ に限定される．また，各受理ラベルは一度だけ伝播処理の対象となり，そこから生成される子ラベルの数は有限である．さらに，単純パスの辺数は高々 $|V| - 1$ であるため，ラベルの生成を繰り返してパスを延長できる回数も有限である．したがって，初期ラベルから生成されるラベルの総数は有限であり，各 $U[v]$ へのラベル追加回数，およびそれに伴うキュー Q へのノード挿入回数も有限であるため，WP-LC は有限回の反復で停止する． □

命題 4.2. 始点 $s = v_1$ から任意のノード v_p への単純パス $P = (v_1, \ldots, v_p)$ およびその前半部分路

$$P_i = (v_1, \ldots, v_i) \qquad (i = 1, \ldots, p)$$

について，WP-LC の実行中，以下の少なくとも 1 つが起きる．

(i) P に対応するラベル l が生成される．

(ii) ある $i \in \{1, \ldots, p\}$ について，IsPruned によって P_i に対応するラベルが枝刈りされる．

(iii) ある $i \in \{1, \ldots, p\}$ について，CanDominate によって P_i に対応する候補ラベルが棄却されるか，既存ラベルが $L[v_i]$ から除去される．

証明. 単純パス $P = (v_1, \ldots, v_p)$ を考え，ある $i \in \{1, \ldots, p\}$ について (ii)(iii) のどちらか起きた場合，命題 4.2 の主張は満たされる．以下では (ii)，(iii) のいずれも起こらない場合のみを考え，(i) が必ず起こることを数学的帰納法で示す．

まず，$P_1 = (v_1)$ に対応する初期ラベル l_{start} は Algorithm 1 の 3 行目の初期化において生成され，$L[s]$ および $U[s]$ に追加される．次に，k を $p-1$ 以下の任意の自然数として，$P_k = (v_1, \ldots, v_k)$ に対応するラベル l_k が生成されたと仮定し，$P_{k+1} = (v_1, \ldots, v_{k+1})$ に対応するラベル l_{k+1} が生成されることを示す．(iii) は起こらないと仮定しているので，l_k は CanDominate に基づいて削除されず，未伝播ラベルとして有限反復後に伝播処理の対象となる．その際, Algorithm 1 は ノード v_k から出るすべての有向辺に対して緩和を試みるため，辺 (v_k, v_{k+1}) に対しても緩和が行われる．また，P は単純パスであるから $v_{k+1} \notin V(P_k)$ であり，CheckAndRecordCycle によってこの緩和が打ち切られることはない．さらに，(ii) は起こらないので，$P_{k+1} = P_k + (v_k, v_{k+1})$ に対応するラベルの生成は IsPruned によって枝刈りされない．また，(iii) は起こらないので，UpdateWeakParetoSet において P_{k+1} に対応する候補ラベルが CanDominate によって棄却されることもない．なお，UpdateWeakParetoSet 5 行目の重複判定によって候補ラベルが棄却される場合には，P_{k+1} に対応する同一ラベルが既に生成されている．したがって，$P_k = (v_1, \ldots, v_k)$ に対応するラベル l_k が生成されたという仮定の下で，P_{k+1} に対応するラベル l_{k+1} が生成される．以上により，帰納的に (i) が成り立つことが示された． □

命題 4.3. 関数 CanDominate に基づくラベルの削除操作によって，s-t 間の弱パレートパスが失われることはない．

証明. ノード v におけるラベル $l_{\text{win}}, l_{\text{lose}}$ について

$$l_{\text{win}} \prec_{\text{str}} l_{\text{lose}}$$

が成り立ち，CanDominate$(l_{\text{win}}, l_{\text{lose}}, v)$ が true を返したとする．l_{win} および l_{lose} の親ノードを辿っていくことにより得られる単純 s-v パスを，それぞれ P_{win}，P_{lose} とする．なお，P_{win} と P_{lose} に対応するラベル l_{win}，l_{lose} は事前に関数 CheckAndRecordCycle で閉路を含まないことを確認しているため，関数 CanDominate の実行時点では単純パスである．また，ラベルの支配関係から

$$\boldsymbol{c}(P_{\text{win}}) < \boldsymbol{c}(P_{\text{lose}}) \tag{4.6}$$

が成り立つ．

l_{lose} を削除することで s-t 間の弱パレートパス P^* が失われると仮定する．このとき，

$$P^* = P_{\text{lose}} + P_{vt}$$

と書ける．ここで，P_{vt} は P^* の v-t 部分路である．P^* は単純パスであるため，P_{vt} も単純パスである．まず，$v \neq t$ であることを示す．$v = t$ と仮定すると，$P^* = P_{\text{lose}}$ であるため，P_{lose} は弱パレートパスである．しかし，(4.6) より，P_{lose} は P_{win} に強支配されるため，矛盾である．よって $v \neq t$ が成り立つ．

次に，ラベル $l_{\text{win}}, l_{\text{lose}}$ のパス上に既に現れた発散ノード集合 H_{win} と H_{lose} を考える．具体的な定義は (4.5) を参照されたい．まず，$v \in V_{\text{div}}$ の場合には，CanDominate が上で仮定

したように true を返している，かつ $v \neq t$ であることから

$$H_{\text{win}} \subseteq H_{\text{lose}} \tag{4.7}$$

が成り立つ．一方，$v \notin V_{\text{div}}$ の場合には，(4.1) より，v に至るパス上に発散ノードは存在しないため，

$$H_{\text{win}} = H_{\text{lose}} = \emptyset$$

である．したがって，常に (4.7) が成り立つ．

ここで，

$$P' = P_{\text{win}} + P_{vt}$$

を考える．P' は一般には単純パスとは限らないため，閉路が含まれる一般の場合を考える必要がある．まずは

$$P' \text{ に含まれる任意の閉路 } C \text{ について } c^1(C) \geq 0 \text{ かつ } c^2(C) \geq 0 \tag{4.8}$$

が成り立つことを背理法で示す．P' に含まれるある閉路 C' が

$$c^1(C') < 0 \text{ または } c^2(C') < 0 \tag{4.9}$$

を満たすと仮定する．s-v パス P_{win} と v-t パス P_{vt} はそれぞれ単純パスであるから，$P' = P_{\text{win}} + P_{vt}$ に閉路 C' が形成されるためには，結合点 v 以外に P_{win} と P_{vt} に共通ノードが存在する．したがって，単純パス P_{vt} を構成するノード集合を $V(P_{vt})$ とすると，

$$x \in \textsc{Path}(l_{win}) \cap V(P_{vt}) \cap V(C') \quad (x \neq v)$$

をとることができて，(4.9) より $x \in V_{\text{div}}$ であり，ゆえに $x \in H_{\text{win}}$ である．この事実と (4.7) より $x \in H_{\text{lose}} \subseteq \textsc{Path}(l_{\text{lose}})$ が得られる．したがって，$x \in \textsc{Path}(l_{\text{lose}}) \cap V(P_{vt})$ である．このことから，パス $P^* = P_{\text{lose}} + P_{vt}$ はノード x を少なくとも 2 回通ることになるが，これは P^* が単純パスであることに矛盾する．したがって，(4.8) が成り立つ．

得られた性質 (4.8) から，s-t パス P' に含まれる閉路を除去してもコストは増加しない．したがって，パス P' から閉路除去を繰り返して最終的に得られる単純 s-t パス $\widehat{P}$ について

$$\boldsymbol{c}(\widehat{P}) \leq \boldsymbol{c}(P')$$

が成り立つ．一方，(4.6) より

$$\boldsymbol{c}(P') = \boldsymbol{c}(P_{\text{win}}) + \boldsymbol{c}(P_{vt}) < \boldsymbol{c}(P_{\text{lose}}) + \boldsymbol{c}(P_{vt}) = \boldsymbol{c}(P^*)$$

となる．よって，

$$\boldsymbol{c}(\widehat{P}) \leq \boldsymbol{c}(P') < \boldsymbol{c}(P^*)$$

が得られ，単純 s-t パス $\widehat{P}$ が P^* を強支配することが示される．これは P^* が弱パレートパスであるという仮定に矛盾する．

以上より，CanDominate に基づくラベルの削除操作によって，弱パレートパスが失われることはないことが示された． □

命題 4.4. 関数 IsPruned による枝刈り操作によって，s-t 間の弱パレートパスが失われることはない．

証明. 関数 IsPruned による枝刈り操作により，ある弱パレートパス P^* が失われたと仮定する．枝刈りが行われた時点の IsPruned の引数 v について，P^* を s-v パス P_{sv} と v-t パス P_{vt} に分割する．すなわち

$$P^* = P_{sv} + P_{vt}$$

であり，$\mathbf{new_cost} = \boldsymbol{c}(P_{sv})$ を満たす．

IsPruned で枝刈りが行われるのは，関数内の以下の 3 か所いずれかで true が返されるときである．

(i) $d_v^1 = \infty$ または $d_v^2 = \infty$ を満たす．(疑似コード 4〜5 行目)

(ii) $\boldsymbol{w} < \boldsymbol{est}$ を満たす．(疑似コード 8〜9 行目)

(iii) ある $l' \in L[t]$ について $l'.\mathbf{cost} < \boldsymbol{est}$ を満たす．(疑似コード 12〜13 行目)

3 つすべてのケースについて，true が返されて枝刈りが行われると矛盾が生じることを示す．事前探索で設定した各変数の定義は 4.1 節も参照のこと．

まず (i) のケース，すなわち $d_v^1 = \infty$ または $d_v^2 = \infty$ となる場合を考える．すべての辺コストは有限な実数であるため，v から t へのパスが存在すれば，そのパスの両目的関数値は有限である．したがって，d_v のいずれかの成分が ∞ であるならば，v から t へのパスは存在しない．しかし，これは P^* の後半部分路である v-t パス P_{vt} が存在することに矛盾する．

次に，(ii) のケースを考える．$P_{vt} \in \mathcal{P}_{vt}^{\leq n-1}$ であることと，式 (4.2) より，

$$\boldsymbol{d}_v \leq \boldsymbol{c}(P_{vt})$$

が成り立つ．したがって，

$$\boldsymbol{est} = \boldsymbol{c}(P_{sv}) + \boldsymbol{d}_v \leq \boldsymbol{c}(P_{sv}) + \boldsymbol{c}(P_{vt}) = \boldsymbol{c}(P^*)$$

より，$\boldsymbol{est} \leq \boldsymbol{c}(P^*)$ である．一方，4.1.2 節で構成した単純 s-t パス P_1 は，式 (4.4) より $\boldsymbol{c}(P_1) \leq \boldsymbol{w}$ を満たす．したがって，$\boldsymbol{w} < \boldsymbol{est}$ が成り立つ場合，

$$\boldsymbol{c}(P_1) \leq \boldsymbol{w} < \boldsymbol{est} \leq \boldsymbol{c}(P^*)$$

となる．これは，単純 s-t パス P_1 が P^* を強支配することを意味し，P^* が弱パレートパスであることに矛盾する．

最後に，(iii) のケースを考える．すなわち，

$$l'.\mathbf{cost} < \boldsymbol{est}$$

が成り立つ $l' \in L[t]$ が存在する．ラベル l' に対応する単純 s-t パスを P'' とすると，

$$\boldsymbol{c}(P'') = l'.\mathbf{cost} < \boldsymbol{est} \leq \boldsymbol{c}(P^*)$$

であるため，単純 s-t パス P'' は P^* を強支配し，P^* が弱パレートパスであることに矛盾する．

以上より，IsPruned の枝刈りにより弱パレートパス P^* が失われたと仮定すると，枝刈りが行われる可能性がある (i)-(iii) いずれのケースでも矛盾が生じる．よって，IsPruned の枝刈り操作によって，弱パレート s-t パスが失われることはない． □

これまでの議論から，命題 4.1 より WP-LC は有限回の反復で必ず停止する．さらに，任意の弱パレートパス P^* を考える．命題 4.2 において，P^* の前半部分路が IsPruned または CanDominate によって失われるならば，それぞれ命題 4.4，4.3 に矛盾する．したがって，命題 4.2 (i) より，P^* に対応するラベルが必ず生成され，アルゴリズム終了時まで $L[t]$ に保存されている．

一方，弱パレートパスではない単純 s-t パス P を考える．BSP(s,t) の実行可能解集合は有限であるため，強支配関係を辿ることで，P を強支配する弱パレートパス $\widehat{P}$ が存在する．終点 t では CanDominate がパス履歴の包含条件によらず強支配されたラベルの削除を許可するため，P に対応するラベルは最終的な $L[t]$ には残らない．したがって，アルゴリズム終了時の $L[t]$ は BSP(s,t) の弱パレートパス集合に一致する．

よって，WP-LC の正当性に関する以下の定理を得る．

定理 4.1. WP-LC は有限回の反復後に必ず停止し，アルゴリズム終了時の $L[t]$ は BSP(s,t) の弱パレートパス集合に一致する．

4.4. 計算量の解析

本節では，提案する WP-LC の最悪計算量について議論する．ノード v のラベル集合 $L[v]$ に追加されるラベルの総数を N_v とし，

$$N_{\max} = \max_{v \in V} N_v$$

とする．ここで，N_v は最終的に $L[v]$ に保存されているラベルだけでなく，$L[v]$ から削除されるラベルも含むことに注意する．

定理 4.2 (WP-LC の最悪計算量)**.** WP-LC の最悪時間計算量は $O(|V||A|N_{\max}^2)$ である．

証明. WP-LC の実行時間は，事前探索と本探索の計算量の和によって決まる．まず，事前探索（Forward PreSearch および Backward PreSearch）では，各目的関数について単目的の最短路アルゴリズム（SPFA 等）を適用する．Forward PreSearch では，各目的について Bellman–Ford 法を実行し，負閉路の影響を受けるノードを到達可能性の探索によって特定するため，計算量は $O(|V||A|)$ である．Backward PreSearch では，各目的について SPFA または Bellman–Ford 法を 1 回ずつ，計 2 回アルゴリズムを実行して下界・上界を求める．SPFA と Bellman–Ford 法の計算量は $O(|V||A|)$ であるため，事前探索全体の計算量は $O(|V||A|)$ である．

次に本探索について評価する．探索中，各ノード $u \in V$ において $L[u]$ に追加されるラベルの累積個数は高々 $N_{\max}$ である．これらのラベルは，ノード u から出る各隣接辺 $\mathrm{arc} = (u,v)$ を通じて後続ノード v へと伝播される．したがって，アルゴリズム全体を通したラベルの緩和処理の総実行回数は，最大で $|A|N_{\max}$ 回となる．

1 回のラベル緩和処理（ノード u のラベル l をノード v へ伝播し，新たなラベル l_{new} を評価する処理）において，以下の各ステップが実行される．

1. **枝刈り判定 (IsPruned)**: 終点 t で保持されているラベル集合 $L[t]$（サイズは最大で $N_{\max}$）との強支配か否かを判定する．第 1 目的による二分探索で比較対象を絞り込むが，探索候補の第 1 目的コストが大きく，かつ第 2 目的コストが小さい場合など，絞り込まれた比較範囲が $L[t]$ 全体に及ぶことがあり，最悪の場合はすべてのラベルとベクトル比較を行うため $O(N_{\max})$ の時間を要する．

2. **閉路検知** (CheckAndRecordCycle): ラベルの親ラベルを始点 s まで遡ってパスを復元・照合する．単純パスの制約によりパスの最大長は $|V|-1$ に制限されるため，この処理の計算量は $O(|V|)$ である．
3. **ラベルの削除可否判定と更新** (UpdateWeakParetoSet): $L[v]$ との間で強支配か否かの判定および包含関係を判定する．新しいラベル l_{new} と既存ラベル l' との間で関数 CanDominate が呼び出された場合，各々のパスに含まれる発散ノード集合（要素数は最大 $|V|$）の包含関係 $H_{\text{win}} \subseteq H_{\text{lose}}$ を検証する必要がある．この包含判定は，一方の集合の要素をリストとして持ち他方を線形探索する実装では $O(|V|^2)$ を要するが，本実装ではノード番号を添字とする真偽値配列を用いて，まず H_{lose} のパス上の発散ノードにフラグを立て（$O(|V|)$），続いて H_{win} のパス上の発散ノードについてそのフラグを配列の直接参照により確認する（各ノードにつき $O(1)$，全体で $O(|V|)$）という手順を踏む．フラグの除去にかかる計算量も $O(|V|)$ である．このため，1 回の CanDominate 全体は $O(|V|)$ で実行できる．これを $L[v]$ の各要素に対して行うため，更新処理全体での最悪計算量は $O(|V|N_{\max})$ となる．

以上の各処理は直列に実行されるため，1 回のラベル緩和にかかる最悪計算量は $O(N_{\max} + |V| + |V|N_{\max}) = O(|V|N_{\max})$ となる．なお，キューへの挿入戦略である SLF 拡張にかかる計算量も，未伝播ラベル集合 $U[v]$ の走査であるため $O(N_{\max})$ であり，このオーダーに吸収される．全体のラベル緩和回数 $O(|A|N_{\max})$ に，1 回あたりの計算量 $O(|V|N_{\max})$ を乗じることで，本探索の総計算量は $O(|V||A|N_{\max}^2)$ となる．なお，上記の評価はラベル配列の要素移動を除くものであり，各ラベルの挿入・削除がそれぞれ高々 1 回であることから，要素移動の総計算量は本探索の計算量に吸収される．したがって，事前探索の計算量 $O(|V||A|)$ と本探索の計算量 $O(|V||A|N_{\max}^2)$ を合わせると，アルゴリズム全体の最悪計算量は $O(|V||A|N_{\max}^2)$ と結論付けられる． □

5. 数値実験

本節では，数値実験により WP-LC の有用性を検証する．数値実験では，先行研究である Sedeño-Noda ら [13] が提案した BDijkstra 法，および弱パレートパスを全列挙するように BDijkstra 法を改良した WP-LS との比較実験を行い，計算時間を評価した．なお，計算時間には事前探索の時間も含める．本研究における実験環境は以下の通りである．本コードは `https://github.com/Jiwoong0107/WP-LC-Algorithm` より利用可能である．

OS: Windows 11 Home

CPU: 11th Gen Intel(R) Core(TM) i5-1155G7 @ 2.50GHz (2.50 GHz)

RAM: 8.0 GB

実装言語: C++17

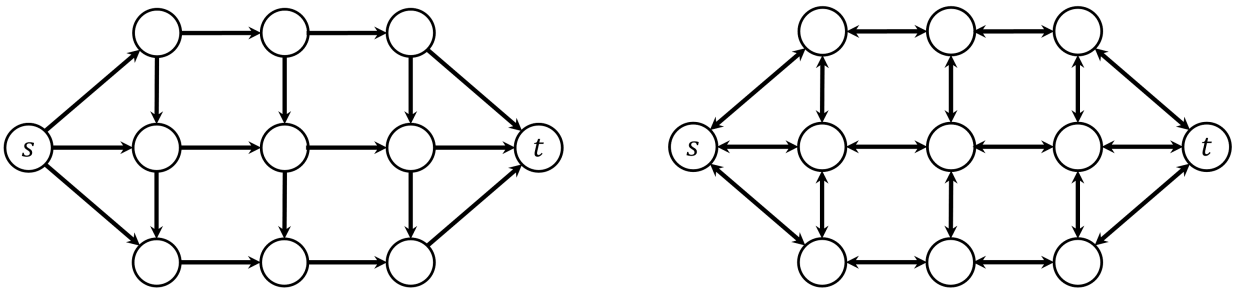
なお，弱パレートパスの探索においてはノードごとのラベル保持数が膨大となり，動的なメモリ確保の余分な処理が計算時間に深刻な影響を与える．そのため本実装では，各ノードのラベル管理に可変長配列（std::vector）を用い，不要となったラベルは，対応するノードの保持ラベル集合から除外する一方，確保済みの配列容量は解放せず，後続のラベル追加時に再利用する．これにより，探索中に発生する動的メモリ確保の回数を抑制している．

5.1. インスタンスの設定

本研究では，アルゴリズムの性能を多角的に評価するため，以下の 2 種類のグラフをインスタンスとして用いた．

グリッドグラフ

右および下方向への有向辺のみを持つ非巡回型グリッドグラフと，上下左右に有向辺を持つ一般的なグリッドグラフの 2 種類を採用した．辺コストの設定として，非巡回型では負の値を含む条件とし，一般的なグリッドグラフでは非負の条件と負の値を含む条件の両方を検証した．なお，具体的な辺コストの範囲は，非負の場合は 1 から 10 の整数値，負の値を含む場合は -10 から 10 の整数値とした．本実験では，アルゴリズムの解の個数の増大に対する耐性を評価するため，比較的小規模なものから段階的にサイズを拡大した 4 種類のグリッドグラフを生成した．一般に，グリッドグラフは多数の迂回路が存在する構造上の特性から，パレートパスおよび弱パレートパスの数がグラフサイズの拡大に伴って指数関数的に増大しやすいことが知られており，多目的最短路アルゴリズムの限界性能を測るベンチマークとして適している．いずれのネットワークにおいても，先行研究に倣い，始点 s はグリッド外の独立したノードとして左端の全ノードへ有向辺を持ち，終点 t は右端の全ノードから有向辺を受ける構造とした．3×3 の非巡回型グリッドグラフの例を図 6 (i) に，一般的なグリッドグラフの例を図 6 (ii) に示す．



(i) (ii)

図 6: 3×3 グリッドグラフの例 (i) 非巡回型 (ii) 一般型

大規模道路ネットワーク

アメリカの都市の道路網をモデル化したグラフであり，DIMACS Shortest Path Implementation Challenge [4] のベンチマーク問題を利用した．各インスタンスの詳細は表 1 に示す通りである．また，大規模道路ネットワークでは，各ネットワークについて 50 組の異なる始終点を用いた．なお，本実験で用いる –NY は，NY の各ノード v に区間 $[0, 20000]$ の整数上の一様分布に従うポテンシャル π_v を与え，各辺 (u, v) の第 2 目的のコストを $c^2_{uv} - \pi_u + \pi_v$ に変更したデータセットである．この際，先行ノードと後続ノード間のポテンシャルの値の差分を用いているため，閉路を周回した際に各辺に加えられるポテンシャル差の総和は 0 となり，閉路の総コストは変化せず，新たな負閉路が生成されないことが保証される．

5.2. 実験結果

実験結果の各表の各アルゴリズムの列は実行時間（秒）を表し，各項目に対する Avg., Min, Max は，それぞれ平均値，最小値，最大値である．実行時間はグリッドグラフ，大規模道路ネットワークともに 3600 秒で打ち切っている．Solved/n の行は，n 個のインスタンス中，各アルゴリズムが制限時間内に弱パレートパスまたはパレートパスを全列挙できたインスタンス数を示す．また，N_t^{wp}, N_t^p は，それぞれ弱パレートパス及びパレートパスの数を

表 1: 大規模道路ネットワークのベンチマークインスタンス詳細

名称	地域	$\|V\|$: ノード数	$\|A\|$: 有向辺数	辺コスト
NY	ニューヨーク市	264,346	733,846	正
COL	コロラド州	435,666	1,057,066	正
FLA	フロリダ州	1,070,376	2,712,798	正
−NY	ニューヨーク市	264,346	733,846	負も含む

表す．N_t^{wp} については WP-LC，N_t^p については BDijkstra が時間内に列挙できたインスタンスのみを対象として算出しており，未解決例は含めない．

5.2.1. グリッドグラフ

表 2 にグリッドグラフの実験結果について示す．注意 4.1 より非正閉路が存在しないインスタンスについては閉路検知の処理を行わない．閉路検知を行わないインスタンスについては，表 2 のインスタンス名の横に「閉路検知なし」と記載してある．本研究では，各サイズについて 5 つのインスタンスで実験を行った．

まず，一般型（正重み）の結果に着目する．グリッド構造の特性上，グラフサイズの拡大に伴い弱パレートパス数（N_t^{wp}）が爆発的に増加している．特筆すべきは，同じ枝刈りロジックを実装しているにもかかわらず，すべてのサイズで WP-LC の計算時間が WP-LS の計算時間よりも短くなった点である．これは，探索順序の管理手法の違いが枝刈りの効率に影響しているためと考えられる．厳密なコスト順にラベルを確定させる WP-LS に対し，WP-LC は拡張 SLF 等のヒューリスティクスによって有望なパスを早期に終点まで探索する性質を持つ．その結果，探索の初期段階で終点において，弱パレートパスとして有望なラベルが獲得され，これがグリッド内に存在する膨大な未確定ラベルに対する強力な枝刈り基準として機能する．つまり，WP-LC は質の高い暫定解を早期に発見することで，WP-LS よりも実質的な探索空間を効果的に削減できていると言える．

次に，負重みを含むインスタンス（非巡回型および一般型）について考察する．ラベル設定法に基づく WP-LS や BDijkstra は非負辺を対象とする実装を用いており，負辺に対応するための再重み付けは行っていないため，表中の該当箇所は評価対象外とし空欄としている．非巡回型（負重みあり）において，WP-LC は 100×100 サイズまでの全インスタンスを解決できたものの，150×150 サイズでは 5 つのインスタンスのうち制限時間内に解決できたものは 1 つのみであった．弱パレートパスの数を比べると，100×100 の 5 つの問題例では平均 2440.2 個なのに対し，150×150 の解決できた 1 つの問題例では 10201 個となっている．グラフ規模の拡大に伴い，弱パレートパス数や探索過程で保持するラベル数が増加したことが，150×150 サイズの一部のインスタンスでタイムアウトした一因となった可能性がある．さらに，一般型（負重みあり）では，5×5 という小サイズのインスタンスにおいて最終的なパス数が平均 32.2 本と極めて少ないにもかかわらず，計算時間が 8.87 秒に達している．また，6×6 のインスタンスでは 1 つも解くことができなかった．これは，一般グラフにおいて負重みが混在することで，2 目的最短路問題特有の閉路が頻発するためである．グラフ中に負重みを含むことで，探索中に閉路を形成する候補が多数発生し，CheckAndRecordCycle による親ラベルの走査が頻繁に実行されたことが，計算時間増大の一因であると考えられる．

表 2: グリッドグラフにおける計算結果

		N_t^{wp} (個)	WP-LC (秒)	WP-LS (秒)	N_t^{p} (個)	BDijkstra (秒)
一般型 (正重み：$1 \sim 10$)・閉路検知なし						
50×50	Solved/5	—	5/5	5/5	—	5/5
	Avg.	168.6	**0.07**	0.10	47.6	0.02
100×100	Solved/5	—	5/5	5/5	—	5/5
	Avg.	963.2	**4.18**	6.02	118	0.25
150×150	Solved/5	—	5/5	5/5	—	5/5
	Avg.	3362.4	**79.32**	219.73	196.4	1.07
200×200	Solved/5	—	5/5	5/5	—	5/5
	Avg.	7635.2	**1321.39**	1555.36	301	3.05
非巡回型 (負重みあり：$-10 \sim 10$)・閉路検知なし						
50×50	Solved/5	—	5/5	—	—	—
	Avg.	589.8	**0.70**	—	—	—
100×100	Solved/5	—	5/5	—	—	—
	Avg.	2440.2	**83.40**	—	—	—
150×150	Solved/5	—	1/5	—	—	—
	Avg.	10201	**3398.8**	—	—	—
一般型 (負重みあり：$-10 \sim 10$)・閉路検知あり						
4×4	Solved/5	—	5/5	—	—	—
	Avg.	11.4	**0.022**	—	—	—
5×5	Solved/5	—	5/5	—	—	—
	Avg.	32.2	**8.87**	—	—	—
6×6	Solved/5	—	0/5	—	—	—
	Avg.	—	**3600**	—	—	—

平均時間は，未解決のインスタンスの計算時間を 3600 秒として算出している．

5.2.2. 大規模道路ネットワーク

表 3 に大規模道路ネットワークにおける各アルゴリズムの実行結果を示す．なお，いずれのインスタンスにおいても非正閉路が存在しないため，本節の全数値実験において閉路検知の処理を行わないようにしている．

まず，パス数の違いに着目する．表 3 の N_t^{wp} と N_t^{p} を比較すると，すべてのネットワークについて弱パレートパスの数がパレートパスの数を大幅に上回っている．NY の平均パレートパス数が 158.86 本であるのに対して，平均弱パレートパス数は 608.58 本に増加しており，最も規模の大きい問題である FLA では，平均パレートパス数が 361.02 本であるのに対して，WP-LC が解決した 43 例における平均弱パレートパス数は 2412.70 本に増加している．特に Max を見ると，NY で 15,040 本，FLA で 17,050 本に達しており，弱パレートパスの探索空間がいかに膨大であるかを示している．

表 3: 大規模道路ネットワークの実験結果

		N_t^{wp} (個)	WP-LC (秒)	WP-LS (秒)	N_t^{p} (個)	BDijkstra (秒)
NY	Solved/50	—	50/50	49/50	—	50/50
	Avg.	608.58	**7.91**	95.07	158.86	0.95
	Min	1	0.84	0.24	1	0.24
	Max	15040	124.07	251.85	844	6.54
COL	Solved/50	—	48/50	47/50	—	50/50
	Avg.	1098.63	**259.21**	486.84	296.3	8.43
	Min	1	2.37	0.39	1	0.39
	Max	11625	3488.22	3171.61	2564	112.11
FLA	Solved/50	—	43/50	37/50	—	50/50
	Avg.	2412.70	**724.38**	1352.61	361.02	25.26
	Min	10	18.38	1.08	10	0.96
	Max	17050	2573.10	3430.08	1279	246.02
−NY	Solved/50	—	50/50	—	—	—
	Avg.	608.58	**7.90**	—	—	—
	Min	1	0.88	—	—	—
	Max	15040	115.87	—	—	—

平均時間は，未解決のインスタンスの計算時間を 3600 秒として算出している．
最大計算時間は解決されたインスタンスの中での最大値である．

次に，弱パレートパスを全列挙する 2 つのアルゴリズム（WP-LC と WP-LS）の性能を比較する．表 3 が示す通り，比較対象とした NY，COL，FLA の 3 ネットワークでは，WP-LC は WP-LS より平均実行時間が短く，制限時間内に解決したインスタンス数も多かった．NY においては，WP-LS の平均計算時間が 95.07 秒であるのに対し，WP-LC は 7.91 秒と大幅な高速化を実現している．さらに，経路探索の規模が大きく解集合が膨大な COL や FLA においても WP-LC は WP-LS より多くのインスタンスで制限時間内に解決しており，平均実行時間も約半分に抑えられている．ここで特筆すべきは，グリッドグラフでの実験結果と比較して，大規模道路ネットワークにおいて WP-LC の優位性がより顕著に現れている点である．グリッドグラフでは WP-LC の WP-LS に対する高速化は最大でも約 2.8 倍に留まったのに対し，道路ネットワークの NY では，未解決インスタンスの計算時間を 3600 秒として算出した平均実行時間が，WP-LC では 7.91 秒，WP-LS では 95.07 秒と，両者の間に約 12 倍の差がみられた．両手法はともに支配関係に基づく同等の枝刈り処理を実装しているが，探索順序の管理手法に決定的な違いがある．WP-LS に代表される厳密なコスト順のラベルの緩和は，大規模道路ネットワークのように探索空間が大きいインスタンスでは，優先度付きキューの更新が計算負荷の増大につながる．対照的に，ラベル修正法に基づく WP-LC は，拡張 SLF 等の適用により厳密な順序付けを緩め，有望なパスを深部まで早期に探索する性質を持つ．これにより，WP-LC は他を強支配しやすい強力なラベルを探索の初期段階

で獲得できるため，結果として同一の枝刈り条件であっても，実ネットワーク空間においてはより連鎖的に無駄な探索を削減できていると推察される．

さらに，表 3 に示された –NY の実験結果について考察する．前節で述べた通り，–NY はポテンシャルを用いて非正閉路を持たない負辺を含むように設計されたネットワークである．本研究で比較に用いた WP-LS および BDijkstra は，負閉路がなければ再重み付けによって負辺を扱えるが，本論文ではこの処理を実装していないため，–NY については比較対象外とした．ラベル修正法を採用する WP-LC は，この –NY においても 50 インスタンスすべてを制限時間内に解決している．通常の NY と –NY の結果を比較すると，ポテンシャルの性質上，始点から終点までの全パスに対して同一の定数（終点と始点のポテンシャル差）が加算されるのみでパス間の支配関係は変化しないため，最終的なパス数（N_t^{wp}）は理論通り完全に一致している．さらに，本実験で用いた –NY インスタンスでは，負辺を含むにもかかわらず，通常の NY とほぼ同等の実行時間が得られた．

最後に，厳密なパレートパスのみを探索する BDijkstra との比較を行う．実験の結果，BDijkstra は全インスタンスを制限時間内に解決し，最も難易度の高い FLA でも平均 25.26 秒と極めて高速であった．この速度差は，最終的なパス数（N_t^p）の少なさと，ノードの再処理が不要なラベル設定法の性質に起因する．対照的に WP-LC は，負辺や閉路に対応するため再処理を伴うラベル修正法を採用しており，微小なトレードオフを持つ膨大な弱パレートパスを全列挙する計算量的に不利な条件下にある．しかし表 3 が示す通り，WP-LC は二分探索による支配判定の高速化，ラベル配列の確保済み容量の再利用による動的メモリ確保の抑制，拡張 SLF といったアルゴリズム上の工夫により，WP-LS を上回る性能を示した．結論として本実験結果は，適切なデータ構造とヒューリスティクスを組み合わせることで，弱パレートパスの列挙問題に対し，すべてのインスタンスを解くには至らなかったものの，現実的な時間内で計算可能な限界を大幅に押し広げられることを実証した．

5.2.3. NY インスタンスにおける弱パレートパスの考察

本節では，NY のインスタンスを対象に，得られた弱パレートパスの性質について考察する．図 7 は WP-LC で求めた各弱パレートパスの 2 目的関数値をプロットしたものであり，図 8 はそれらのパスを実際の地図上で重ね合わせて可視化したものである．本インスタンスにおいて，Objective 1 は「距離」，Objective 2 は「時間」に対応する．また，図 8 の配色はパスの特性を反映しており，ピンク色に近いほど距離が短い（Objective 1 最小化寄り）パス，緑色に近いほど時間が短い（Objective 2 最小化寄り）パスを表している．

図 7 から読み取れる最大の特徴は，弱パレートパスの分布が大きく 2 つのクラスタに二極化している点である．これを図 8 の地図上で確認すると，パスの地理的分布も同様に，ピンク色のパス群と緑色のパス群にはっきりと分かれていることがわかる．具体的な道路利用状況と照らし合わせると，緑色のパス群は距離が長くなる代わりに速度が速い高速道路などを利用するルートであり，対照的にピンク色のパス群は，距離は短いが信号や渋滞の影響を受けやすい一般道などを走行するパスである．ここで特筆すべきは，図 7 において左上のグループ（距離優先）と右下のグループ（時間優先）の間に，解が存在しない広範な空白領域が存在することである．これは，対象とする s-t 間において，距離と時間のトレードオフが連続的ではなく，極端な二者択一の構造になっていることを示唆している．すなわち，「距離をわずかに犠牲にして，時間を少しだけ短縮する」といった中間的な妥協案となるパスは構造的に存在しにくい．

さらに，図 7 の右下の時間優先グループにおける解の分布に着目すると，Objective 1（距

離）が増加しても Objective 2（時間）の減少幅が極めて小さい解が多数密集していることが確認できる．これは，高速道路を主軸とするルートにおいて，利用するインターチェンジや接続道路の微細な選択の違いによって，同程度のパフォーマンスを持つ弱パレートパスが多数生成されているためと考えられる．

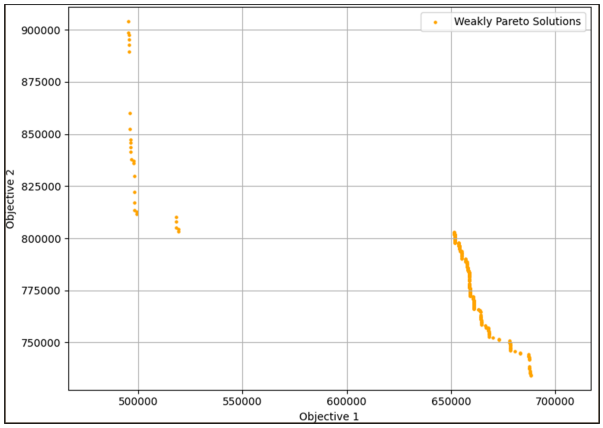


図 7: 各 s-t 弱パレートパスの目的関数値のプロット (NY)

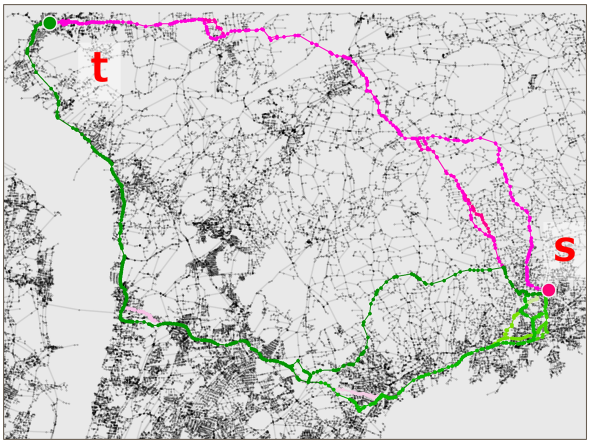


図 8: s-t 弱パレートパス (NY)

6. まとめ

本研究では，移動時間や運賃などトレードオフの関係にある 2 つの目的を持つ 2 目的最短路問題において，通常のパレートパスだけでなく，柔軟な意思決定に役立つ弱パレートパスを列挙する手法について論じた．特に，既存研究の多くが辺コストの非負性や負閉路の不在を仮定していたのに対し，本研究では負のコストや多様な閉路構造を含む一般的な有向グラフを扱える新たなアルゴリズム WP-LC の提案および評価を行った．

本研究の主な成果として，ラベル修正法に基づく WP-LC を提案した．WP-LC は，探索過程における動的な閉路検知・分類機能に加え，事前探索で得た下界と上界を用いて有望でない探索を早期に打ち切る枝刈り戦略（IsPruned），推定費用ベクトルを用いた拡張 SLF による優先探索，および発散ノードの包含関係に基づく安全なラベル削除機構（CanDominate）を備えている．これにより，解の列挙を阻害する非正閉路が存在する場合であっても無限ループを回避して探索を継続し，弱パレートパスをすべて列挙できることを理論的に証明した．

さらに，提案手法の有用性を検証するため，グリッドグラフおよび大規模な道路ネットワークを用いて Dijkstra 法ベースの拡張手法（WP-LS）との比較実験を行った．その結果，WP-LC は拡張 SLF や強力な枝刈りによって無駄な探索空間を大きく削減し，解が爆発的に増大する大規模な道路ネットワークにおいても WP-LS より大幅に速く動作することを確認した．特に，NY ネットワークにおいては，未解決インスタンスの計算時間を 3600 秒として算出した平均実行時間において，WP-LC と WP-LS の間に約 12 倍の差がみられた．さらに，本研究で比較に用いたラベル設定法の理論保証の前提を満たさない負辺を含む -NY に対しても，WP-LC が全 50 インスタンスを制限時間内に解決することを確認した．

一方で，実験結果からは今後の課題も明らかになった．WP-LC の計算量はノード数および探索中に生成されるラベル数に強く依存しており，特に負の辺コストが混在する条件下では，閉路検知のために親ラベルを遡る処理が頻発するため，これが計算時間増大の一因と

なっている可能性がある．したがって，閉路検知のさらなる高速化や計算量オーダーの改善が重要な課題である．加えて，弱パレートパスは通常のパレートパスに比べてパスの個数が顕著に増大する傾向があるため，実用的な時間内で有用な解を抽出するためのフィルタリングや，得られた弱パレートパス間の類似度の検討なども，今後の課題として挙げられる．

参考文献

[1] R. Bellman: On a routing problem. *Quarterly of Applied Mathematics*, **16** (1958), 87–90.

[2] J. Craveirinha, J. Clímaco, R. Girão-Silva, and M. Pascoal: Multiobjective path problems and algorithms in telecommunication network design—overview and trends. *Algorithms*, **17** (2024), 222.

[3] E.W. Dijkstra: A note on two problems in connexion with graphs. *Numerische Mathematik*, **1** (1959), 269–271.

[4] DIMACS: 9th DIMACS implementation challenge—shortest path. `http://www.dis.uniroma1.it/challenge9/download.shtml` (accessed January 11, 2026).

[5] D. Duque, L. Lozano, and A.L. Medaglia: An exact bidirectional method for the bioobjective shortest path problem. *Computers & Operations Research*, **62** (2015), 1–11.

[6] F. Guerriero and R. Musmanno: Label correcting methods to solve multicriteria shortest path problems. *Journal of Optimization Theory and Applications*, **111** (2001), 589–613.

[7] P. Hansen: Bicriterion path problems. In G. Fandel and T. Gal (eds.): *Multiple Criteria Decision Making Theory and Application* (Springer-Verlag, Berlin, 1980), 109–127.

[8] Y. Kergosien, A. Giret, E. Neron, and G. Sauvanet: An efficient label-correcting algorithm for the multiobjective shortest path problem. *INFORMS Journal on Computing*, **34** (2022), 76–92.

[9] E.Q.V. Martins: On a multicriteria shortest path problem. *European Journal of Operational Research*, **16** (1984), 236–245.

[10] E.F. Moore: The shortest path through a maze. In H.H. Aiken (ed.): *Proceedings of an International Symposium on the Theory of Switching, Part II* (Harvard University Press, Cambridge, 1959), 285–292.

[11] J.L.E. Santos: Uma abordagem ao problema do trajecto óptimo multiobjectivo. *Investigação Operacional*, **19** (1999), 211–226.

[12] V.N. Sastry, T.N. Janakiraman, and S.I. Mohideen: New algorithms for multi objective shortest path problem. *OPSEARCH*, **40** (2003), 278–298.

[13] A. Sedeño-Noda and M. Colebrook: A biobjective Dijkstra algorithm. *European Journal of Operational Research*, **276** (2019), 106–118.

[14] F.B. Zhan and C.E. Noon: Shortest path algorithms: An evaluation using real road networks. *Transportation Science*, **32** (1998), 65–73.

南川 智都
静岡理工科大学 情報学部 情報学科

〒 437-0032 静岡県袋井市豊沢 2200-2
E-mail: minamikawa.norito@sist.ac.jp